



An intelligent memetic approach to detect online fraud for distributed fintech environments

Saad M. Darwish¹ · Amr Ibrahim Salama² · Adel A. Elzoghbi³ ·
Noha A. El-Shoafy⁴

Received: 20 January 2025 / Accepted: 27 September 2025
© The Author(s) 2025

Abstract

Detecting fraudulent trading activities in the FinTech sector presents significant challenges due to the highly dynamic nature of financial markets and the rapid adaptation strategies employed by fraudsters. Conventional machine learning techniques, especially when trained on imbalanced datasets, often exhibit classification bias toward the majority class (legitimate transactions), reducing their ability to identify fraudulent behaviors. To address this limitation, we introduce a hybrid memetic-ABC framework that combines Artificial Bee Colony (ABC) sampling for global exploration with memetic local refinement to generate high-quality synthetic minority-class samples guided by feature importance. ABC ensures broad coverage of potential fraudulent patterns, while the memetic refinement fine-tunes synthetic samples and feature subsets to enhance discriminative power, capturing subtle anomalies in transaction amounts, locations, and timing. The framework further incorporates an anomaly detection module that constructs behavioral profiles for traders using historical transaction data, clustering them based on similar trading patterns. This enables identification of key indicators of potential fraud, such as sudden volume fluctuations, irregular transaction timings, and deviations from behavioral norms. For real-time monitoring and scalable deployment in decentralized environments, the system is extended into a Distributed Intrusion Detection System (DIDS) architecture, simultaneously detecting transactional anomalies and network-level threats, including DDoS attacks and unauthorized access attempts. By correlating anomalous trading patterns with DIDS-generated alerts, the approach achieves enhanced detection accuracy and robustness against complex fraud scenarios. Experimental evaluations demonstrate that the proposed model outperforms state-of-the-art techniques, achieving approximately 10% improvement in accuracy.

Keywords E-commerce fraud detection · FinTech security analytics · Swarm intelligence-based sampling · Imbalanced financial transaction data · Distributed anomaly detection system

Extended author information available on the last page of the article

1 Introduction

Fintech forensics refers to the practice of investigating and analyzing financial technology (FinTech) systems, transactions, and data to uncover potential fraud, misconduct, or irregularities. With the rapid adoption of technologies such as online banking, mobile payments, blockchain, and digital currencies in the financial sector, there is an increasing demand for specialized expertise to detect, investigate, and prevent fraudulent activities and security breaches within these systems [1]. Fintech forensics professionals may utilize a range of techniques and tools, including data analytics, digital forensic analysis, and machine learning algorithms, blockchain analysis, and cybersecurity measures, to identify suspicious patterns or anomalies in financial data. They may also work closely with law enforcement agencies, regulatory bodies, and cybersecurity experts to gather evidence and mitigate risks associated with financial crimes such as money laundering, identity theft, and cyber fraud [2]. Fintech forensics solutions use advanced analytics and machine learning algorithms to detect and prevent various types of fraud, including credit card fraud, payment fraud, insider fraud, and account takeover attacks.

Fintech forensics, particularly in the context of detecting online fraudulent activities in different applications as illustrated in Fig. 1, involves employing various techniques and technologies to identify and mitigate fraudulent transactions or behaviors within Fintech systems. The commonly used approaches in Fintech forensics for detecting online fraud include [1–4]: (1) Data Analytics and Machine Learning: These models can identify patterns indicative of fraudulent behavior, such as unusual spending patterns, transactions deviating from typical user behavior, or suspicious IP addresses. (2) Fraud Detection Rules: These rules can be configured to trigger alerts for transactions that meet certain criteria, such as large transactions to new or unusual locations, transactions made at odd hours, or transactions involving high-risk mer-

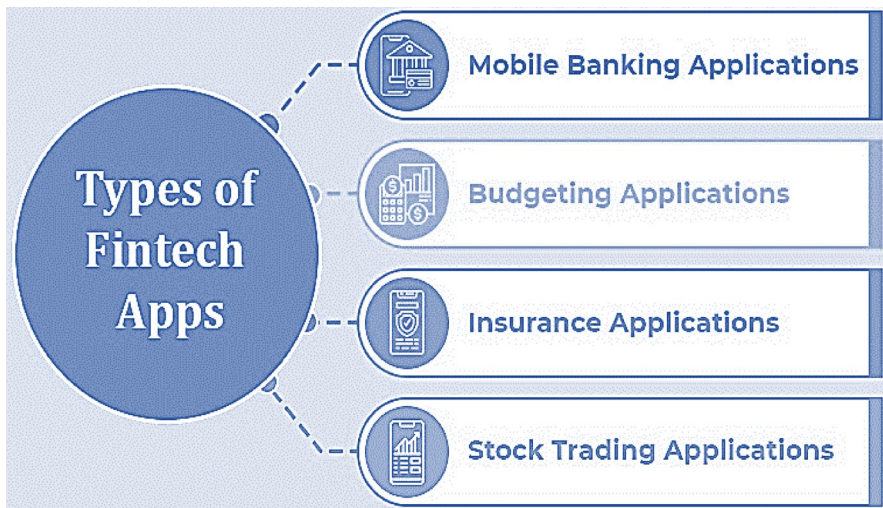


Fig. 1 Types of Fintech Apps

chants. (3) Identity Verification: Advanced identity verification techniques, including biometric authentication or document verification, can further enhance security and reduce the risk of fraudulent activity. (4) Transaction Monitoring and Anomaly Detection: Anomaly detection algorithms can identify deviations from expected patterns and flag potentially fraudulent activity for further investigation. By combining these techniques and leveraging advanced technologies, Fintech companies can develop robust fraud detection systems capable of identifying and mitigating various forms of online fraudulent activities, safeguarding both their platforms and their users' assets.

FinTech forensics encounters multiple challenges in accurately identifying fraudulent activities within financial technology platform [1, 3]. (1) Fraudsters continuously evolve their tactics to bypass security measures and exploit vulnerabilities in FinTech systems. Traditional fraud detection methods may not be sufficient to keep up with the rapidly changing landscape of fraudulent activities. (2) Many FinTech services operate in real-time, processing transactions instantly. Detecting fraud in real-time presents a significant challenge as forensic analysis must occur within milliseconds to prevent fraudulent activities from occurring. (3) In fraud detection, the number of legitimate transactions (non-fraudulent) far outweighs the number of fraudulent transactions. This class imbalance can skew the performance of machine learning models, leading to high false negative rates (missed detections) and low precision. (4) Fintech systems are subject to concept drift, where the statistical properties of the data change over time. Imbalanced data exacerbates this issue, as the distribution of fraudulent transactions may evolve differently from legitimate transactions, leading to model deterioration if not continuously monitored and updated [4]. By carefully navigating these challenges, Fintech companies can develop effective fraud detection models that leverage user behavior analysis to protect their platforms and customers from fraudulent activities.

Handling extremely imbalanced data is a common challenge in machine learning, especially in scenarios where one class greatly outnumbers the others. Several techniques can be used to address this issue, each with its own advantages and disadvantages [5–9]: (1) Undersampling: type of resampling techniques that helps balance the dataset by reducing the size of the majority class; it can lead to faster training times and less memory usage. However, it could increase the risk of overfitting, especially when reducing the dataset size significantly. (2) Oversampling: type of resampling techniques that helps balance the dataset by replicating instances of the minority class. It could increase the training time and memory usage, especially when oversampling significantly. (3) Synthetic Minority Over-sampling Technique (SMOTE): type of resampling techniques that generates synthetic samples for the minority class, addressing the imbalance without exact replication. Still, it may introduce noise or unrealistic data points, particularly in high-dimensional spaces and could be sensitive to the choice of nearest neighbors and other hyperparameters.

There are two main techniques under the umbrella of algorithmic-based handling of extremely imbalanced data: (1) Cost-sensitive Learning allows algorithms to assign different misclassification costs to different classes. Still, it requires careful calibration of cost parameters, which may be non-trivial. (2) Ensemble techniques (e.g., Balanced Random Forest, Easy Ensemble) that combines predictions from multiple models, which can help in handling class imbalance, and can improve gen-

eralization performance and robustness. Yet, it increases computational complexity due to training multiple models and requires careful selection of ensemble methods and tuning of hyperparameters. Under the umbrella of algorithm-specific techniques, there are two main approaches: (1) Class weighting that allows algorithms to assign different weights to different classes during training. Yet, it requires tuning of weight parameters, which may be sensitive to the dataset and algorithm. (2) Anomaly detection techniques that focuses on identifying rare instances, which can be beneficial in highly imbalanced datasets. Nevertheless, it assumes that the minority class instances are anomalies, which may not always hold true and requires careful feature engineering and parameter tuning. See [10, 11] for a comprehensive survey regarding these techniques.

Evolutionary techniques, such as genetic algorithms (GAs) and evolutionary strategies (ES), can play a significant role in handling extremely imbalanced data by providing alternative approaches to traditional machine learning methods. These techniques are inspired by biological evolution and natural selection and are particularly useful in complex optimization problems [12]. Evolutionary techniques can optimize sampling strategies for imbalanced datasets, including oversampling, under-sampling, and hybrid approaches like SMOTE. By evolving a population of sampling strategies, these methods can search for the most effective ways to balance the class distribution while minimizing the risk of overfitting or loss of information. Besides, evolutionary algorithms can be used to construct ensembles of classifiers specifically tailored to handle imbalanced data. By evolving a population of diverse classifiers and combining their predictions using techniques such as bagging or boosting, these methods can improve generalization performance and resilience to class imbalance. Overall, the choice of technique depends on the specific characteristics of the dataset, the algorithm used, and the computational resources available. It's often beneficial to experiment with multiple approaches and evaluate their performance using appropriate metrics before selecting the most suitable technique for handling extremely imbalanced data.

1.1 Problem statement

In recent years, the financial technology (FinTech) sector has experienced explosive growth, revolutionizing how individuals and businesses manage their finances. While FinTech innovations have brought about numerous benefits, they have also introduced new challenges, particularly in terms of detecting and preventing financial crimes (e.g. fraudulent activities). Traditional forensic techniques often fall short in addressing the complexities of digital financial transactions, leaving gaps that can be exploited by criminals. Therefore, there is an urgent need to develop advanced techniques and systems for detecting and mitigating online fraudulent trading to ensure the stability and fairness of financial markets.

Detecting online fraudulent trading involves the use of various techniques and technologies to identify suspicious activities within trading platforms or financial systems. For pattern recognition paradigm, the objective is to analyze trading patterns to identify anomalies that may indicate fraudulent activities. Challenges include distinguishing between legitimate fluctuations and fraudulent behavior, especially in

highly volatile markets. In machine learning, the aim is to employ algorithms to learn from historical trading data and detect patterns indicative of fraud. Challenges include the need for large and diverse datasets for training, as well as the constant adaptation of models to evolving fraud tactics. In behavioral analysis approaches, the intended is to monitor user behavior to detect deviations from normal trading activities. Challenges include distinguishing between intentional fraud and legitimate changes in user behavior, such as adapting to market conditions or trying new trading strategies. The objective of transaction monitoring is to implement systems to track and analyze transactions for suspicious activity, such as unusually large trades or frequent transfers between accounts. Challenges include processing large volumes of transactions in real-time and minimizing false positives. Imbalanced data can lead to bias in machine learning models, where the model may become overly biased towards the majority class (legitimate trades) and perform poorly in detecting the minority class (fraudulent trades). This can result in a high rate of false negatives, where fraudulent activities go undetected.

1.2 Research questions

The increasing complexity of online fraudulent trading in FinTech necessitates a focused and systematic investigation. To address the core challenges highlighted in the problem statement, we have refined our research questions to concentrate on three critical areas: class imbalance, behavioral analysis, and model adaptability. These questions form the foundation for designing and evaluating the proposed fraud detection framework.

- **RQ1:** How can advanced sampling techniques, such as ABC-sampling, address class imbalance and improve fraud detection performance in FinTech datasets?

Rationale: Fraudulent transactions represent a minority in real-world datasets, creating challenges for machine learning models. This question focuses on how effective sampling strategies can generate balanced and informative datasets to enhance detection accuracy.

- **RQ2:** What are the most significant behavioral and transactional features that effectively distinguish fraudulent from legitimate trading activities, and how can clustering techniques enhance anomaly detection?

Rationale: By combining feature analysis with unsupervised learning, this question examines the most relevant behavioral indicators and evaluates how clustering can support the identification of abnormal trading patterns.

- **RQ3:** How effective and adaptable is the proposed ABC-sampling and anomaly detection framework compared to traditional fraud detection models in dynamic financial markets? Rationale: This question assesses both the comparative performance and adaptability of the proposed framework, ensuring it remains robust against evolving fraud tactics and volatile market conditions.

1.3 Motivation of the work

Detecting online fraudulent trading in FinTech forensics remains challenging, primarily due to the highly imbalanced nature of transaction datasets. In real-world environments, fraudulent activities represent only a small fraction of total transactions, making their accurate detection difficult. The problem is further compounded by the evolving strategies of fraudsters, which limit the availability of sufficiently labeled data for training robust detection models, especially for rare and emerging fraud patterns. Although several methods have been proposed to handle imbalanced datasets, including oversampling, undersampling, and ensemble techniques, these approaches often incur high computational costs and are less efficient for large-scale online trading platforms, thereby constraining real-time detection capabilities. Another critical challenge lies in feature selection, as identifying discriminative attributes that accurately capture fraudulent behavior is difficult within dynamic and complex trading environments. Insufficient or poorly chosen features can lead to an underrepresentation of fraud patterns, resulting in reduced detection performance. Recently, optimization-based algorithms have gained attention for their ability to enhance the effectiveness and efficiency of fraud detection models under imbalanced data conditions, achieving improved accuracy while simultaneously reducing false positives and false negative.

1.4 Theoretical and managerial contribution

1.4.1 Theoretical contributions

This study enriches the theoretical literature on IDS, anomaly detection, and FinTech fraud detection by introducing a novel integration of the ABC optimization algorithm with an intelligent fraud detection framework specifically designed for highly imbalanced financial datasets. Unlike conventional IDS and anomaly detection models that rely on static resampling or fixed thresholding techniques, this research contributes a dynamic, behavior-driven adaptive sampling mechanism that improves the detection of rare fraudulent activities. The study advances IDS theory by incorporating individualized behavioral profiling and clustering techniques, enabling the detection of subtle deviations from normal trading behavior that traditional IDS approaches often overlook. In the domain of anomaly detection, it extends existing knowledge by combining supervised and unsupervised learning strategies, thereby enhancing the capability to identify complex and evolving fraud patterns in volatile markets. Finally, within the FinTech literature, the study provides a robust framework for real-time fraud detection in decentralized financial ecosystems, demonstrating how adaptive learning and clustering-based profiling can strengthen security and reliability in online trading environments.

1.4.2 Managerial contributions

From a managerial perspective, this study offers actionable insights for FinTech organizations and decision-makers in financial markets. The proposed framework enables

managers to build individualized behavioral profiles for traders and detect anomalies in real time, improving the efficiency and effectiveness of fraud monitoring systems. By employing adaptive sampling and dynamic anomaly detection, organizations can proactively identify emerging fraud patterns and reduce false alarms, which leads to better allocation of resources and improved operational resilience. Moreover, the proposed framework equips managers with tools to maintain robust defense mechanisms in decentralized trading infrastructures, ensuring sustained trust and compliance with evolving regulatory requirements.

1.5 Extension of scholarly discourse on anomaly detection in fintech

The study contributes to the scholarly discourse of anomaly detection in FinTech by emphasizing behavior-centric and swarm intelligence-driven fraud identification strategies. Unlike traditional models that focus solely on transaction attributes, this research introduces custom behavioral analysis to capture nuanced and contextual trading behaviors indicative of fraud. This enriches the academic narrative by highlighting the importance of user-specific profiling and dynamic detection mechanisms in detecting fraudulent patterns, particularly in decentralized and high-volume financial environments. Moreover, the integration of DIDS with financial anomaly detection creates a multidisciplinary intersection between cybersecurity and fraud analytics—paving the way for holistic threat detection approaches in FinTech.

1.6 Contribution to the academic literature of metaheuristics

From the perspective of metaheuristics, the study significantly enhances the academic literature by applying ABC—an evolutionary optimization technique inspired by natural swarm behavior—to both sampling and fraud detection. The research addresses fundamental challenges in imbalanced datasets and dynamic environments, such as concept drift (fraudsters continually evolve their tactics to evade detection) and sample informativeness. By utilizing ABC for adaptive sampling and undersampling optimization, the study showcases how metaheuristics can go beyond parameter tuning to actively guide data preparation and model learning. This elevates ABC's role from a mere optimization tool to a strategic component of intelligent decision systems, demonstrating its robustness, simplicity, and versatility in handling noisy, complex, and evolving data environments.

1.7 Benefits for fintech organizations

FinTech organizations can benefit immensely from the proposed model in several ways. First, the dynamic behavioral profiling and real-time anomaly detection enable early identification of suspicious trading patterns, reducing the risk of financial loss and reputational damage. Second, the model's scalability and integration with DIDS ensure it is suitable for decentralized FinTech platforms that must contend with both transactional fraud and cybersecurity threats simultaneously. Third, the use of adaptive ABC-based sampling helps maintain model performance over time by addressing class imbalance and concept drift—common challenges in live trading environments.

This ensures sustained model accuracy, reliability, and trustworthiness, which are essential for regulatory compliance and customer retention.

1.8 Benefits for other technology industries

Beyond FinTech, other technology industries—such as e-commerce, cybersecurity, telecommunications, and health informatics—can also benefit from this study. The ABC-based adaptive sampling and behavior-driven anomaly detection framework can be generalized to detect anomalies in any domain where imbalanced data and evolving patterns are prevalent. For instance, in cybersecurity, the model can be used to detect abnormal user activities and insider threats. In e-commerce, it could help identify fraudulent purchases or bot activities. In healthcare, it could monitor anomalous patterns in patient data or insurance claims. The flexibility and low parameter tuning requirements of ABC make it highly adaptable for various industry applications where expert knowledge or clean datasets may be limited.

The structure of the paper is organized as follows: Sect. 2 reviews the relevant literature on existing approaches for detecting online fraudulent trading. Section 3 describes the proposed methodology, focusing on handling extremely imbalanced datasets within FinTech forensics applications. Section 4 presents the experimental setup, details the performance evaluation, and discusses the results obtained. Finally, Sect. 7 concludes the study and highlights potential directions for future research.

2 Related work

Detecting online fraudulent trading involves a multifaceted approach encompassing various methodologies aimed at identifying suspicious patterns and minimizing financial risks. These methodologies include rule-based systems, machine learning models, anomaly detection, network analysis, and hybrid techniques (see Fig. 2) [13–20].

- *Rule-based systems* Rule-based systems rely on explicitly defined conditions and thresholds to flag suspicious activity. While effective for known fraud patterns, they often lack adaptability. Sarkar et al. [14] implemented a rule-based system that monitored online financial transactions using predefined behavior triggers such as transaction frequency and amount thresholds. Their work demonstrated strong detection capabilities in static environments but noted significant limitations in identifying evolving fraud patterns without manual rule updates.
- *Machine learning (ML) models* Machine learning models—particularly supervised and deep learning approaches—have emerged as robust tools for detecting fraud based on learned patterns. Pan [16] explored various ML algorithms for fraud detection, emphasizing their scalability and adaptability to real-time transaction streams. Rajendran et al. [17] conducted a comparative study of machine learning (ML) and deep learning (DL) techniques, revealing that deep learning models offer higher detection accuracy but face challenges in explainability.

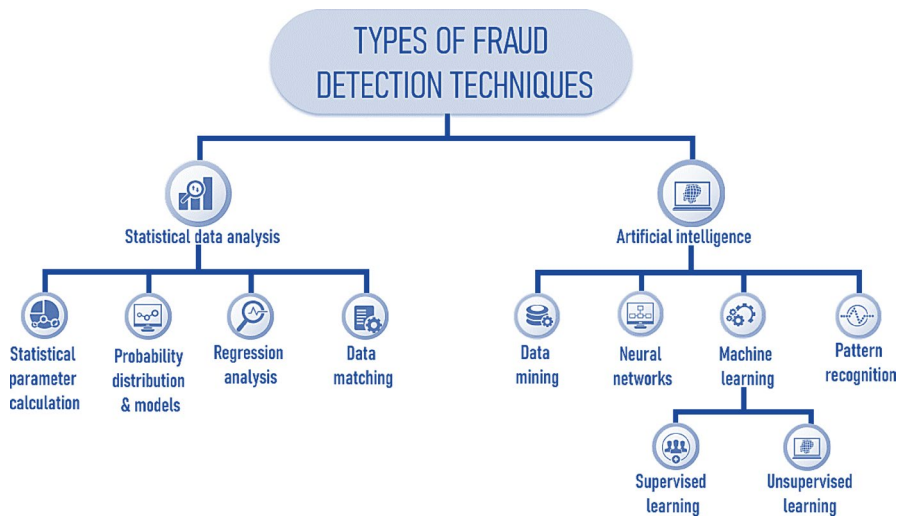


Fig. 2 Types of Fraud Detection Techniques

Similarly, Yi et al. [18] suggested a novel ML framework for fraud detection in capital markets, incorporating domain-specific features and automated learning to improve classification accuracy. Complementing these efforts, Gao et al. [19] provided a comprehensive review of ML applications in finance, identifying both current achievements and future opportunities, particularly regarding explainability, fairness, and real-time deployment.

- *Anomaly detection* Anomaly detection techniques are vital for identifying unusual behaviors without prior knowledge of fraud patterns. Singh et al. [13] conducted a systematic literature review on the use of blockchain-integrated anomaly detection systems for financial fraud. Their work highlighted how decentralized audit trails combined with anomaly scoring mechanisms can enhance transparency and resilience in fraud detection systems. In addition, Dhone and Nitya [20] applied big data analytics and unsupervised learning models to detect rare and emerging fraudulent patterns in financial transactions, focusing on reducing false negatives in high-volume datasets.
- *Network analysis* Network-based fraud detection methods analyze relational patterns between entities to uncover coordinated fraud. Yu et al. [15] proposed a group-based fraud detection network using graph-based algorithms to uncover collusive behaviors on e-commerce platforms. By analyzing transaction linkages and proximity to known fraudulent accounts, their approach significantly improved the detection of organized fraud rings, demonstrating the value of contextual and relational analysis in fraud detection systems.
- *Hybrid approaches* Combining multiple methodologies can enhance detection accuracy and system robustness. For instance, Rajendran et al. [17] highlighted the

effectiveness of integrating deep learning with traditional rule-based systems to capture both known and unknown fraud behaviors. Likewise, Pan [16] proposed a hybrid architecture that fuses supervised ML models with anomaly detection modules, resulting in improved precision and reduced false alarms. These hybrid frameworks strike a balance between interpretability and adaptability, which is essential for real-world financial environments.

Several studies have explored advanced techniques for detecting online fraudulent trading and financial anomalies. In Ref [14]., an IDS was suggested using time-frequency-based features modeled through 2D monetary processes, combined with a random forest classifier and clustering-based hyperparameter tuning to enhance detection accuracy. Ref [21]. introduced an unsupervised, scalable framework that converts real-time trades into graph-based representations, using density signals and a graph topology–time–behavior metric to identify fraudulent traders. The Egret Swarm Optimization Algorithm (ESOA) was applied in Ref [22]. alongside machine learning models with a cost-sensitive objective function for fraud detection. Similarly, Ref [23]. introduced a memory compression-enhanced machine learning framework using a non-linear autoencoding approach to address imbalanced datasets. Ref [24]. tackled challenges like label noise, feature imbalance, and transaction representation by introducing spatial-temporal features, self-supervised objectives, and intelligent sampling to improve classification. To automate fraud detection, Ref [25]. presented the Just-Add-Data (JAD) system, an AutoML framework that selects algorithms, tunes hyperparameters, and evaluates models on highly imbalanced datasets without manual configuration. Furthermore, Ref [26]. recommended a graph neural network (GNN)-based model utilizing subgraph generators, neighborhood sampling, and a self-attention module to better leverage hidden transactional patterns and enhance fraud detection accuracy.

In Ref [27]., a credit card-not-present fraud detection and prevention (CCFDP) framework was suggested, integrating both a Fraud Detection Process (FDP) and a Fraud Prevention Process (FPP) to enhance security against Card-Not-Present (CNP) fraud using big data analytics. The FDP employs five advanced techniques: logistic regression learning (LRL), singular value decomposition (SVD), principal component analysis (PCA), t-distributed stochastic neighbor embedding (t-SNE), and random undersampling to handle dataset imbalance and reduce feature dimensionality, while the FPP acts to prevent malicious activities detected in real time. Ref [28]. introduced a credit card fraud forensic detection model based on Long Short-Term Memory (LSTM) deep neural networks combined with an attention mechanism and uniform manifold approximation to extract significant predictive features from sequential transaction data, enabling banks to effectively minimize fraudulent activities. Similarly, Ref [29]. offered a hybrid approach integrating a resampling technique to handle class imbalance, the XGBoost classifier for accurate prediction, and the Differential Evolution (DE) algorithm for optimal hyperparameter selection, resulting in improved detection performance. In Ref [30]., the authors employed a genetic algorithm (GA) to optimize hyperparameters and compared its efficiency with grid search (GS), applying machine learning models such as support vector machines (SVM), decision trees (DT), logistic regression (LR), and random forests (RF). GA

outperformed GS by achieving higher accuracy, precision, recall, and F1-score while reducing computational time, using undersampling to address data imbalance. Ref [31]. recommended an automated detection framework utilizing the Bat Optimization Algorithm (BOA) for selecting discriminative features, followed by the SMOTE to balance skewed datasets, and a convolutional neural network (CNN) enhanced with a full center loss function for anomaly detection. Experimental results demonstrated superior performance, achieving approximately 99% detection accuracy on the Kaggle credit card fraud dataset.

In Ref [32]., the authors presented BalanceGAN, a generative adversarial network-based framework designed to handle highly imbalanced datasets for online banking fraud detection, where synthetic data from the generator is used to train the detection model and later fine-tuned through transfer learning on real-world datasets, effectively preventing model overfitting. Similarly, Ref [33]. introduced a hybrid method combining autoencoders (AE) and fully connected deep networks (FCDN), where one AE learns fraudulent patterns, another reduces dimensionality, and an FCDN classifies encoded data, further enhanced using the SMOTE. In Ref [34]., the authors employed XGBoost and fully connected neural networks to improve the detection of complex fraudulent activities in network-based financial transactions. Ref [35]. explored Quantum Machine Learning (QML) techniques by evaluating four models, demonstrating that the Quantum Support Vector Classifier achieved the highest F1-score of 0.98 for both fraud and non-fraud detection, while other models like the Variational Quantum Classifier, Estimator Quantum Neural Network (QNN), and Sampler QNN also showed promising results, though efficient quantum algorithms and larger datasets remain a challenge. Ref [36]. presented three complementary models: a fraud risk prediction model, an economic optimization framework, and a machine learning-based fraud detection model, achieving up to 52% reduction in expected losses while maintaining a low false-positive rate of 0.4%. Additionally, Ref [37]. compared thirteen statistical and machine learning approaches on both real-world and synthetic payment card transaction datasets, showing that aggregated and genetically optimized features significantly improve fraud detection accuracy. For a broader overview of supervised, unsupervised, optimization-based, and hybrid techniques for banking fraud detection, detailed surveys are available in Refs [38–51].

2.1 Intrusion detection systems (IDS) in distributed fintech environments

In the context of FinTech, where applications operate across cloud platforms, APIs, and microservice architectures, traditional centralized IDS are no longer sufficient. FinTech systems demand distributed IDS (DIDS) that can monitor and respond to threats in real-time across multiple nodes and environments. Shrivastava and Chaturvedi [59] review various machine learning-based IDS models specifically designed for distributed networks and emphasize the growing importance of adaptive detection in decentralized digital infrastructures, such as those used in FinTech. Distributed Denial of Service (DDoS) attacks are particularly relevant to FinTech because they can cripple real-time transaction systems, delay or block financial operations, and compromise customer trust. For instance, a DDoS attack targeting a trading platform during market hours could obstruct trade execution, cause financial losses, or even be used as a distraction for coordinated

fraud. Hijazi et al. [60] suggested a hybrid classification-based adaptive DIDS for local area networks, showing its effectiveness in handling volumetric attacks like DDoS, which are increasingly common against financial institutions.

Moreover, FinTech applications often operate over federated environments where data privacy regulations limit centralized data collection. In this regard, Khraisat et al. [61] survey Federated Learning (FL)-based IDS, which distribute model training across edge devices without sharing sensitive data, offering a privacy-preserving and scalable solution for FinTech fraud and intrusion detection. Cloud-native FinTech platforms are also susceptible to lateral movement attacks and insider threats, requiring IDS solutions that operate across virtualized and containerized infrastructures. Ahmadi [62] compares several IDS frameworks tailored for cloud environments, emphasizing their applicability to sectors like FinTech where elastic scaling and high availability are critical. Finally, Diana et al. [63] provide a comprehensive overview of IDS technologies in network security, stressing the importance of selecting context-aware detection systems. They highlight that IDS must evolve alongside digital transformation trends, such as those observed in FinTech, to address threats that span across devices, services, and platforms.

2.2 The need to extend the related work

Enhancing fraud detection in online trading under imbalanced datasets requires adopting advanced methodologies and data-driven approaches to improve detection accuracy. A promising direction involves deriving novel feature sets from transactional data to uncover subtle behavioral patterns indicative of fraudulent activities. By incorporating custom behavioral analysis, more informative features can be extracted, improving model performance. Additionally, swarm optimization-based sampling techniques, particularly the Artificial Bee Colony (ABC) algorithm, can effectively address class imbalance by optimizing undersampling and synthetic data generation, selecting representative samples while minimizing redundancy compared to conventional methods. ABC's stochastic search mechanism maintains solution diversity, making it robust in noisy or uncertain environments where traditional techniques often fail. Its bee-inspired solution update process enables adaptive learning from both local and global contexts, which is crucial for detecting evolving fraud patterns. Compared to other swarm intelligence approaches like Particle Swarm Optimization (PSO) and Ant Colony Optimization (ACO), ABC achieves a better exploration–exploitation balance, avoids premature convergence, and handles high-dimensional, imbalanced, and noisy financial data more efficiently. Furthermore, ABC requires fewer control parameters, reducing the risk of overfitting and making it suitable for real-world FinTech applications where domain expertise and tuning resources are limited. Its robustness, scalability, and ability to optimize sampling strategies and model training pipelines make ABC a resource-efficient solution for detecting fraudulent activities in decentralized and complex financial ecosystems [52, 53].

To further enhance the proposed fraud detection framework, the concept of DIDS can be integrated to provide real-time, scalable, and context-aware monitoring across the decentralized infrastructure commonly found in FinTech platforms. Incorporating DIDS allows the system to extend its monitoring beyond transactional anomalies to include network-level threats, such as Distributed Denial of Service (DDoS) attacks or unauthorized access patterns, which are often used to mask or facilitate fraudu-

lent trading. By deploying lightweight IDS agents across different nodes (e.g., trading servers, user endpoints, cloud microservices), the system can collect distributed behavioral and network data to detect coordinated fraud attempts or security breaches that are not evident in transaction data alone. The behavioral profiles derived from transactional features can be cross-referenced with alerts from the distributed IDS to improve detection precision and reduce false positives.

3 The proposed model

Effective fraud detection requires modeling an account's typical transactional behavior and identifying deviations indicative of fraudulent activity. This involves selecting key transaction features, constructing behavioral profiles from historical non-fraudulent data, and defining criteria for triggering alerts when anomalies occur. The approach aims to automate user-profiling systems to enable reliable, real-time detection of fraudulent transactions. The proposed framework employs a multi-stage classification strategy for fraud detection, integrating rule-based filtering, K-means clustering, an ABC-based classifier, and a K-Nearest Neighbors (KNN) classifier into a unified pipeline to optimize detection while minimizing false positives. In the first phase, raw transaction attributes pass through a rule-based filter and a level-one K-means classifier that derives critical transaction values. The level-two ABC classifier then refines predictions using a structured dataset of seven Boolean variables, including five critical transaction values, the level-one output, and historical classification data. This hierarchical processing is combined via semantic decision-level fusion, enabling the integrated outputs of K-means and ABC to yield higher-quality predictions and bridge multiple clustering levels with interpretable results. In the second phase, customer profiles are generated from incoming transactions and compared against stored spending profiles using the KNN classifier. Critical aspects of ABC performance, such as the local search mechanism and parameter-driven fitness function, are optimized within the fusion level, enhancing overall fraud detection accuracy and robustness.

3.1 Phase 1: Training phase

The training phase generates an expenditure profile P for each cardholder based on past transactions. This profile is created by applying K-means and ABC-based classification on filtered transaction data. Let T be the transaction data for a particular cardholder, $X = \{x_1, x_2, \dots, x_N\}$ is the raw attributes of a transaction, C_i is the classification result at classifier level i , and P represents the stored expenditure profile of the cardholder (created during the training phase).

3.1.1 Step 1: ABC-sampling for balancing imbalanced datasets

In the context of online trading, fraudulent transactions represent a small portion of the total transaction volume, leading to a highly imbalanced dataset. The challenge is to build a classifier capable of detecting fraud accurately while managing this class imbalance to minimize both false negatives (missed fraud) and false positives (legitimate transactions flagged as fraud). Assume a dataset D containing N transactions,

each represented by $X_i \in \mathbb{R}^d$ that is the feature vector for the i -th transaction, consisting of d attributes (e.g., transaction amount, frequency, user activity metrics) and $y_i \in \{0,1\}$ that is the label for the transaction, where $y_i = 1$ indicates a fraudulent transaction and $y_i = 0$ indicates a legitimate transaction. This dataset can be split into two subsets:

$$D = D_0 \cup D_1, \quad (1)$$

where D_0 consists of legitimate transactions and D_1 consists of fraudulent ones, with $|D_1| \ll |D_0|$, reflecting the high class imbalance. The primary objective is to design a classifier $f: \mathbb{R}^d \rightarrow \{0,1\}$ that maximizes fraud detection accuracy while balancing precision and recall.

To address class imbalance, the model applies ABC-driven swarm optimization to generate a balanced dataset. This technique selects a representative subset $\tilde{D} \subset D$ with enhanced diversity and coverage, achieved by optimizing either undersampling or oversampling:

$$\tilde{D} = \underset{\tilde{D}}{\operatorname{argmax}} (Diversity(\tilde{D}) + Coverage(\tilde{D})), \quad (2)$$

This results in a more balanced training set, ensuring that the model encounters both legitimate and fraudulent behaviors, improving robustness and generalization. Diversity measures how varied the selected subset $\tilde{D}$ is in terms of feature distribution and class representation. A diverse subset should capture a broad range of feature values, transaction types, and patterns present in the original dataset D . Maximizing diversity ensures that the selected subset $\tilde{D}$ includes examples that represent different transaction behaviors, both legitimate and fraudulent. This is particularly important in detecting fraud, as fraudulent behavior can vary widely. Coverage assesses how well the subset $\tilde{D}$ includes critical examples that represent the most relevant patterns and behaviors in the dataset. In fraud detection, coverage is vital to ensure that typical fraudulent transaction profiles are included. Maximizing coverage ensures that the subset $\tilde{D}$ captures essential characteristics or patterns that the classifier will encounter in real-world data, making the model more robust and generalizable.

ABC-based sampling is a robust technique for handling imbalanced datasets by generating synthetic samples to augment minority classes—in this case, fraudulent transactions [52, 53]. Here's how ABC optimization steps can be linked to fraud detection for balancing a dataset in a banking fraud detection system.

- **Initialization** Let $X_i = \{x_{i1}, x_{i2}, \dots, x_{id}\}$ represent a transaction in the minority class (fraudulent). Initialize N food sources from fraudulent transactions as $X = \{X_1, X_2, \dots, X_N\}$ where d is the number of features in each transaction.
- **Fitness function** $F(X)$ The fitness function evaluates how well a synthetic transaction represents a real fraudulent transaction. A common choice is to use a distance metric, such as Euclidean distance, based on the closeness of transaction

features to the original minority class:

$$F(X_i) = \frac{1}{1 + \text{dist}(X_i, \text{Centroid}_{\text{fraud}})} \quad (3)$$

where $\text{Centroid}_{\text{fraud}}$ is the centroid of the fraudulent transactions in the feature space. Higher fitness values indicate more realistic fraudulent transactions.

- *Employed bee phase* Each employed bee modifies the current food source (synthetic transaction) X_i by exploring neighboring values, creating a new transaction V_{ij} as:

$$V_{ij} = X_{ij} + \varphi_{ij}(X_{ij} - X_{kj}) \quad (4)$$

X_{ij} is the j -th feature of the i -th transaction, φ_{ij} is a random number between -1 and 1 , and X_{kj} is a randomly selected transaction from the fraudulent class, $k \neq i$. If the fitness $F(V_{ij}) > F(X_i)$ replaces X_i as a better synthetic fraudulent transaction.

- *Onlooker bee phase* Onlooker bees choose food sources based on their fitness probability P_i .

$$P_i = \frac{F(X_i)}{\sum_{j=1}^N F(X_j)} \quad (5)$$

Higher fitness food sources have a higher probability of being selected for further synthetic transaction generation. This focuses the sampling on more representative fraudulent examples.

- *Scout bee phase* If a food source X_i does not yield improved fitness over a certain number of trials (a parameter known as “limit”), it is abandoned, and a new food source X_i is randomly initialized:

$$X_i = \text{random initialization in feature space} \quad (6)$$

- *Stopping criteria* ABC sampling continues until the minority class size reaches a targeted ratio with respect to the majority class, achieving dataset balance. If N_{fraud} and N_{genuine} are the sizes of fraudulent and genuine classes, respectively, the algorithm stops when:

$$\frac{N_{\text{fraud}}}{N_{\text{genuine}}} \geq \text{Target Ratio} \quad (7)$$

This pseudocode listed in Algorithm 1 provides a structured representation of the ABC Sampling Technique, ensuring diversity and coverage of the synthetic samples to address class imbalance effectively.

Input:

Dataset $D = \{(X_i, y_i) \mid X_i \in \mathbb{R}^d, y_i \in \{0,1\}\}$

D_1 : Minority class (fraudulent transactions)

D_0 : Majority class (legitimate transactions)

Target ratio R_{target}

Parameters: N (number of food sources), $limit$ (max_ trials), ϕ (exploration factor)

Output:

Balanced dataset $\hat{D}$

Algorithm Steps**1- Initialization:**

- Extract D_1 from D (minority class).
- Initialize N food sources $X = \{X_1, X_2, \dots, X_N\}$, where each X_i is randomly selected from D_1 .
- Compute the centroid of D_1 :

$$Centroid_{fraud} = \frac{1}{|D_1|} \sum_{X_i \in D_1} X_i$$

2- Define Fitness Function:

- For each food source X_i

$$F(X_i) = \frac{1}{1 + dist(X_i, Centroid_{fraud})}$$

(e.g., using Euclidean distance).

3- Repeat Until $|D_1|/|D_0| \geq R_{target}$

- **Employed Bee Phase:**

For each food source X_i :

- 1- Generate a new candidate V_{ij}

$$V_{ij} = X_{ij} + \phi_{ij}(X_{ij} - X_{kj})$$

X_k is randomly selected from D_1 , $k \neq i$, and $\phi_{ij} \in [-1,1]$ is a random number.

- 2- Evaluate $F(V_{ij})$:

- **If** $F(V_{ij}) > F(X_i)$, update $X_i = V_{ij}$.

- **Onlooker Bee Phase:**

- 1- Calculate selection probability for each food source X_i

$$P_i = \frac{F(X_i)}{\sum_{j=1}^N F(X_j)}$$

- 2- Select food sources based on P_i .

- 3- Repeat the steps of the employed bee phase for selected sources.

- **Scout Bee Phase:**

For each X_i :

- 1- If X_i does not improve after $limit$ trials:

- Abandon X_i .

- Replace X_i with a new transaction randomly initialized in the feature space.

4- Stopping Criteria:

- Stop the process when the size of the minority class $|D_1|$ satisfies:

$$|D_1|/|D_0| \geq R_{target}$$

5- Combine Data:

Combine the synthetic transactions with the original dataset to form $\hat{D}$.

Return: $\hat{D}$, the balanced dataset.

Algorithm 1: ABC Sampling Technique for Balancing Imbalanced Datasets

The ABC Sampling Technique described in Algorithm 1 can be conceptually linked to the p-best and g-best mechanisms found in swarm optimization algorithms like

PSO, although it follows a different strategy due to its bee-inspired framework. While ABC does not explicitly define p-best (personal best) or g-best (global best), similar dynamics are present: each food source acts as a potential solution, with its fitness value reflecting the quality of that solution, akin to p-best. During the onlooker bee phase, solutions with higher fitness are more likely to be selected, implicitly guiding the swarm toward globally promising regions—functionally similar to g-best exploitation. Furthermore, in the employed bee phase, the generation of new candidate solutions based on the difference between current and randomly chosen food sources introduces a local search mechanism that mimics the learning behavior associated with p-best. Additionally, the centroid of the minority class can be interpreted as a g-best-like anchor, encouraging convergence toward a representative fraud pattern. The scout bee phase, which replaces stagnating solutions, helps maintain population diversity and prevents premature convergence—paralleling strategies in swarm intelligence to balance exploration and exploitation.

In general, ABC algorithm is not restricted to Fintech applications. ABC is a versatile, population-based optimization algorithm inspired by the foraging behavior of honey bees, and it has been effectively applied across a wide range of domains beyond financial fraud detection. For instance, in healthcare, ABC has been used to optimize feature selection for disease diagnosis systems, such as in heart disease prediction or medical image segmentation, where identifying the most relevant diagnostic indicators is critical. In engineering, ABC is employed for structural design optimization, load dispatch problems in power systems, and tuning control parameters in robotics. In wireless sensor networks, ABC optimizes routing and energy-efficient clustering, extending network lifetime. It is also used in bioinformatics for gene selection and protein structure prediction. In image processing, ABC aids in tasks such as image thresholding and edge detection. Moreover, in software engineering, it is used for test case generation and parameter tuning in machine learning models. These diverse applications demonstrate ABC's adaptability and strength in solving complex, multi-dimensional, and nonlinear optimization problems across various fields.

Integrating feature importance into the ABC sampling process significantly improves the quality and relevance of synthetic samples, particularly in critical areas like fraud detection. This strategy allows the algorithm to emphasize features that are most influential in identifying fraudulent transactions, thereby enhancing the accuracy and robustness of the resulting classifier. In fraud detection, features such as transaction amount, time of day, and device type often hold greater predictive power. When all features are treated equally during optimization, the generated synthetic samples may overlook the complex patterns that define actual fraudulent behavior. To address this, the process begins with estimating feature importance using a baseline Random Forest model, valued for its interpretability. This is followed by applying the widely used permutation importance method, a model-agnostic approach that evaluates feature significance by randomly shuffling each feature's values and measuring the resulting decrease in model performance. The outcome is a set of normalized importance scores for each feature, which then guide the ABC algorithm in producing synthetic samples that better represent meaningful patterns. Consequently, instead of using the original Eq. (3) in the ABC fitness function, a modified version

that integrates these importance scores is employed to more accurately direct the generation process toward the most critical aspects of fraudulent behavior.

$$F(X_i) = \frac{1}{1 + \sqrt{\sum_{j=1}^d I_j \cdot (X_{ij} - \text{Centroid}_{\text{fraud},j})^2}} \quad (8)$$

$$I_j \in [0,1], \sum_{j=1}^d I_j = 1 \quad (9)$$

This adjustment ensures that deviations in highly significant features are penalized more, encouraging the generation of synthetic samples that closely align with meaningful patterns in the fraudulent class. In this case, each I_j quantifies the contribution of the j -th feature to the model's predictive performance.

The proposed framework employs a hybrid optimization strategy that integrates ABC sampling with memetic local refinement. ABC is utilized for global exploration, generating diverse synthetic samples for the minority (fraudulent) class based on feature importance, ensuring broad coverage of potential fraudulent patterns. Once promising solutions are identified, the memetic local refinement mechanism fine-tunes the synthetic samples and feature subsets to maximize their discriminative power. This refinement adjusts key attributes, such as transaction amounts, locations, and timing, to better capture subtle anomalies while preserving the integrity of critical features. Through iterative evaluation and improvement, the approach enhances classification metrics, including precision, recall, and F1-score, resulting in high-quality synthetic data that improves the model's robustness, scalability, and accuracy in detecting complex fraud scenarios.

3.1.2 Step 2: Rule-based filtering

The rule-based engine acts as an initial filter in the transaction classification pipeline, identifying potentially fraudulent activities by measuring deviations from the cardholder's normal behavior. It applies both general and customer-specific rules, such as average spending patterns and billing–shipping inconsistencies, to flag suspicious transactions. This preliminary filtering reduces data volume and prepares the dataset for more advanced classification stages [54, 55]. Let X_i^{filtered} denote the transactions passing the filter:

$$X_i^{\text{filtered}} = \{X_i \in D \mid \text{meet criteria}\} \quad (10)$$

The database comprises records of customers' credit card transactions, where each transaction is characterized by a defined set of attributes outlined in Table 1. The selection of these features is supported by the findings in Ref [64], where Darwish (2020) demonstrated that incorporating diverse transactional, behavioral, and contextual attributes significantly enhances the accuracy and robustness of fraud detection systems. Specifically, factors such as transaction location, transaction amount, card usage frequency, and authentication type were shown to be critical in capturing deviations from normal user behavior. The study emphasized the importance of semantic fusion of multiple classi-

fiers, which relies heavily on the discriminative power of well-chosen features—further validating the relevance of the attributes adopted in this work for effective and intelligent fraud detection.

These attributes are processed by a rules engine, which applies a predefined set of rules to analyze the data. Through this analysis, the rules engine generates five key filtered parameters, outlined in Table 2, which are used to calculate critical values for each transaction. These critical values act as indicators for an initial classification, categorizing the transaction as either potentially fraudulent or normal. These rules utilize critical values δ derived from key transaction parameters to provide a preliminary classification as listed in Algorithm 2.

Based on the rule-based filtering logic in Algorithm 2, the estimated proportion of transactions to be removed at this pre-classification stage could range from 30% to 45% of the total transaction data. This estimate assumes that a significant portion of daily customer transactions follow consistent, historically typical behavior—such as low-frequency usage, minimal overdraft activity, spending within expected limits, and usage in a limited number of locations. These transactions are likely to meet the “not fraud” conditions defined by the thresholds and can therefore be safely excluded from further complex analysis. Similarly, highly anomalous transactions that clearly exceed the defined boundaries (e.g., $10\times$ daily spending or $5\times$ location diversity) can be confidently flagged as fraud early on. This dual-path filtering (both legitimate and clearly suspicious cases) allows a substantial portion of data to be filtered out with minimal risk of misclassification.

Table 1 Key attributes for credit card transaction analysis

No.	Attribute	Description
1	Card Id	A unique identifier assigned to each credit card
2	Location of transaction T_{loc}	The geographic location where the transaction occurred
3	Current balance CC_{cb}	The remaining balance available on the card at the time of the transaction.
4	Average bank balance B_{ab}	The average balance maintained by the cardholder in their associated bank account over a certain period
5	Amount of transaction T_{amt}	The monetary value of a single transaction.
6	Credit card age CC_{age}	The duration of time since the credit card was issued
7	Authentication type T_{au}	The method used to authenticate the transaction (e.g., PIN, OTP, biometric).
8	Location of card used CC_{loc}	The geographic location where the credit card is physically used
9	Number of overdraft $N_{cc_{od}}$	The number of instances the cardholder has exceeded their credit limit
10	Overdraft transaction T_{od}	Specific transactions made during overdraft periods
11	Card used T_{cu}	Total number of card use during a period
12	Card used transaction $N_{T_{cu}}$	The total number of transactions conducted using the card

Table 2 Five key filtered parameters for credit card transaction analysis

No.	Attribute	Equation	Definition
1	$CC_{Frequency}$	$\frac{T_{cu}}{CC_{age}}$	This parameter tracks the frequency of credit card usage over a specific time frame (e.g., daily, weekly, or monthly)
2	$CC_{Usage_Location}$	$\frac{CC_{Loc}}{T_{loc}}$	This parameter captures the geographic locations where the credit card is being used.
3	$CC_{Overdraft}$	$\frac{N_{cc_od}}{T_{cu}}$	This parameter measures how often and how quickly the credit limit is exceeded by the cardholder
4	$CC_{Balance}$	$\frac{CC_{db}}{B_{ab}}$	This refers to the current available balance on the credit card, reflecting the funds left after all transactions and payments
5	$CC_{Daily_Spending}$	$(100000 - B_{ab}) \times \left(\frac{CC_{age}}{30} \right)$	This parameter calculates the average amount spent daily using the credit card over a defined period.

Input: Transaction's attributes for each customer $A_{T_{i,j}}$
 The five filtered parameters for each customer's transaction $FP_{T_{i,j}}$
// computed according to Table 2

Output: The initial transaction's classification $\xi_{T_{i,j},L_0}$
// level 0 classification

Begin

- **IF** $CC_{Frequency} < 0.2$ **THEN** not fraud, $\delta_1 = CC_{Frequency}$
// The threshold of 0.2 represents a low frequency of card usage, which is considered typical or non-suspicious behavior for certain cardholders
- **IF** $N_{T_{cu}} > (5 \times CC_{Frequency})$ **THEN** fraud, $\delta_1 = N_{T_{cu}} \times CC_{Frequency}$
// The rule sets a threshold that triggers fraud detection when the current transaction count $N_{T_{cu}}$ exceeds five times the typical frequency
- **IF** $CC_{loc} \leq 5$ **THEN** not fraud, $\delta_2 = 0.01$
// The threshold of 5 indicates that using a card in five or fewer distinct locations is considered normal or typical behavior for the cardholder
- **IF** $T_{loc} > 5 \times CC_{loc}$ **THEN** fraud, $\delta_2 = CC_{Usage_Location}$
// The threshold is set at five times the cardholder's normal location diversity. If the current number of locations exceeds this value, the activity is flagged as suspicious.
- **IF** $CC_{od} \leq 0.2$ **THEN** not fraud, $\delta_3 = CC_{Overdraft}$
// The threshold 0.2 indicates a very low frequency of credit limit exceedance. Values at or below this level suggest that the cardholder has rarely, if ever, gone over their credit limit.
- **IF** $T_{od} = \text{true}$ **THEN** fraud, $\delta_3 = T_{od} \times CC_{Overdraft}$
// suggests the cardholder is spending funds they don't have access to, which could signal fraudulent activity, especially if it deviates from their usual behavior
- **IF** $CC_{cb} \leq 0.25$ **THEN** not fraud, $\delta_4 = CC_{cb}$
// A remaining balance at or below 25% suggests the cardholder has already used a significant portion of their credit limit
- **IF** $T_{od} = \text{true}$ **THEN** fraud, $\delta_4 = CC_{cb} \times CC_{Balance}$
// Check whether overdraft condition occurred today
- **IF** $(10 \times CC_{Daily_Spending}) > T_{amt}$ **THEN** fraud,
 $\delta_5 = T_{amt} / (10 \times CC_{Daily_Spending})$
ELSE not fraud, $\delta_5 = 0.01$
// If a transaction's amount exceeds ten times the cardholder's average daily spending, it is considered an anomaly and flagged as potentially fraudulent

Return $\xi_{T_{i,j},L_0}, \delta_{T_{i,j}} = \{\delta_{1,T_{i,j}}, \delta_{2,T_{i,j}}, \delta_{3,T_{i,j}}, \delta_{4,T_{i,j}}, \delta_{5,T_{i,j}}\}$
// $T_{i,j}$ transaction j for customer i

Algorithm 2: Rule-based filtering (initial classifier)

This 30–45% reduction is justified by the fact that the thresholds used in Algorithm 2 are grounded in behavioral norms and act as high-confidence indicators. For example, thresholds such as a credit card location count of ≤ 5 or daily spending patterns that rarely exceed $10\times$ the norm are based on conservative limits to ensure only well-understood behavior is filtered out. This helps preserve borderline or ambiguous cases for deeper analysis in more advanced models. The actual proportion may vary depending on the transaction profiles of the customer base (e.g., frequent travelers or high-spending users may generate more exceptions), but the algorithm's design

inherently supports early elimination of clearly classifiable cases—thus improving both computational efficiency and model focus on genuinely uncertain or borderline transactions.

The thresholds applied in Algorithm 2—such as the fixed value of 0.2 for $CC_{Frequency}$ or the use of $5\times$ multipliers for metrics like transaction count and location diversity—are static and derived heuristically rather than from adaptive, data-driven methods. While these thresholds may perform reasonably well under specific conditions, they lack the flexibility to accommodate variations across different datasets, customer demographics, regional financial behaviors, and temporal factors such as holidays or economic shifts. For example, what is considered excessive spending or location variance in one region may be perfectly normal in another. If these thresholds are not properly calibrated or regularly updated, the rule-based filtering system risks prematurely misclassifying borderline transactions—labeling legitimate activity as fraudulent or vice versa. Additionally, overly strict or soft thresholds may either fail to reduce the data volume effectively or allow many unnecessary cases to reach downstream classifiers, defeating the algorithm’s intended purpose. Such rigid rules can also introduce systemic bias, unfairly favoring or penalizing certain customer profiles (e.g., frequent travelers or high-volume spenders), thereby compromising the fairness and robustness of the fraud detection system.

3.1.3 Step 3: K-mean classifier (level 1 classification)

In this framework, each transaction is represented by a data structure of five Boolean values, $\delta_{T_{i,j}}$, indicating whether the transaction is classified as fraudulent (suspicious) or legitimate (unsuspicious) based on thresholds defined by the rule-based filtering engine. These preliminary classifications serve as input for the first layer of an advanced system employing k-means clustering, addressing the limitations of rigid rule-based methods in adapting to complex and evolving fraud patterns. The k-means algorithm begins by selecting k initial centroids, with $k = 2$ representing clusters of potentially fraudulent and legitimate transactions. Each transaction, encoded as a point from the Boolean vector, is assigned to the nearest centroid using Euclidean distance to quantify similarity [56, 57]. Cluster centroids are iteratively recalculated as the mean of all points within the cluster, and transactions are reassigned until convergence, producing the trained model output $Model_{T_{i,j}, l1}$. This k-means-based classification enhances the system’s adaptability by dynamically grouping transactions with similar characteristics, overcoming the static nature of rule-based filters [57]. While initial rule-based filtering establishes a baseline, it may misclassify transactions due to fixed thresholds. Introducing clustering provides a data-driven, adaptive layer, improving accuracy and robustness, particularly in detecting sophisticated or emerging fraudulent behaviors that may evade the initial filter.

K-means clustering is preferred in this initial classification stage because it offers a lightweight, unsupervised, and adaptable mechanism for identifying patterns in transactional data that may not be captured by rigid rule-based systems or traditional supervised machine learning classifiers. Unlike baseline ML classifiers such as Random Forest or SVM, which require labeled training data and extensive feature engineering, k-means can operate directly on the output of Boolean rule-based filters

without prior knowledge of class labels. This is particularly advantageous in real-time or early-stage fraud detection, where labeled data may be limited or delayed. Additionally, k-means dynamically groups transactions based on intrinsic similarities, allowing it to detect emergent or evolving fraud behaviors that may not align with historical fraud patterns used to train supervised models. Its simplicity, computational efficiency, and ability to enhance the contextual understanding of suspicious behavior make it a practical and effective choice for this level of classification, serving as a bridge between rigid rule-based detection and more complex downstream classifiers [56, 57].

3.1.4 Step 4: Swarm fraud detection (level 2 classification)

The effectiveness of the K-Means clustering algorithm is highly dependent on the initial selection of centroids, which can often result in suboptimal clustering performance. This limitation is particularly critical in fraud detection applications, where precise classification and accurate anomaly identification are essential. To overcome this challenge, the ABC algorithm is integrated to enhance the clustering process and improve overall detection accuracy. The ABC algorithm optimizes the initialization of centroids and iteratively refines cluster assignments by utilizing the collective intelligence of three types of artificial bees—employed, onlooker, and scout—which collaboratively explore the solution space to identify optimal centroid positions. Using a parameter-driven fitness function, the ABC algorithm ensures the formation of compact and well-separated clusters while simultaneously improving alignment with known classification labels, distinguishing fraudulent transactions from legitimate ones with higher accuracy.

In the adapted approach, each transaction in the dataset is represented as a data structure containing seven Boolean features. These include five critical attributes that encapsulate

key transactional behaviors $\delta_{T_{i,j}} = \{\delta_{1,T_{i,j}}, \delta_{2,T_{i,j}}, \delta_{3,T_{i,j}}, \delta_{4,T_{i,j}}, \delta_{5,T_{i,j}}\}$, along with two additional Boolean values: one representing the classification type computed during the first level of classification $Model_{T_{i,j}, l1}$ (Level 1 classification) and the other indicating the actual classification label derived from historical data in the training set $Model_{real}$. The procedural steps for this enhanced clustering strategy are outlined in Algorithm 3.

1. Initialization:

- Input the dataset D with each transaction represented as a vector of 7 Boolean values: $\left[\delta_{1,T_{i,j}}, \delta_{2,T_{i,j}}, \delta_{3,T_{i,j}}, \delta_{4,T_{i,j}}, \delta_{5,T_{i,j}}, \zeta_{Model,T_{i,j},l1}, \zeta_{Model,real} \right]$
- Define the number of clusters $K = 2$ and the number of artificial bees (employed, onlooker, scout).
- Randomly initialize K centroids C_k within the Boolean feature space.

2. Fitness Function:

Define the fitness function based on:

- Intra-cluster similarity: Minimize the distance between data points and their assigned centroids.
- Classification alignment: Maximize the match between $\zeta_{Model,T_{i,j},l1}$ and $\zeta_{Model,real}$ for each cluster.

$Fitness = \alpha \times Intra - Cluster Similarity + \beta \times Classification Alignment$
 α and β are weighting factors balancing similarity and alignment.

3. Employ Bees:

- Each employed bee explores the neighborhood of its current solution (centroid) to find a better centroid position.
- Update centroids C_k based on the exploration.

4. Calculate Probabilities:

- Calculate the probability P_k of selecting a cluster centroid based on its fitness:

$$P_k = \frac{Fitness}{\sum_{i=1}^k Fitness_i}$$

5. Onlooker Bees:

- Assign onlooker bees to centroids based on P_k .
- Each onlooker bee refines centroids by exploiting promising solutions (clusters with high fitness).

6. Scout Bees:

- Identify stagnating centroids (not improving over iterations) and replace them with new random centroids in the feature space.

7. Cluster Assignment:

- Assign each data point to the nearest centroid using a similarity measure (e.g., Hamming distance for Boolean features).
- Evaluate the clustering result using the fitness function.

8. Termination:

- Repeat Steps 3–7 for a predefined number of iterations or until the centroids converge.
- Output the optimized centroids C_k and the final clusters.

9. Level 2 Classification:

- Based on the final clusters, assign a classification label $\zeta_{Model,T_{i,j},l1}$ for new data points by majority voting of $\zeta_{Model,real}$ values within each cluster.

Algorithm 3: ABC-Based Level 2 Classification for K-Means

3.1.5 Step 5: K-nearest neighbors classifier

In the proposed framework, KNN classifier functions as the final decision-making stage, integrating outputs from Rule-Based Filtering, K-means Clustering (Level 1), and ABC-based Swarm Fraud Detection (Level 2). Initially, Rule-Based Filtering applies predefined rules to eliminate clearly legitimate transactions and flag suspicious ones, reducing computational overhead. K-means Clustering then groups transactions by similarity, assigning preliminary labels, while Swarm Fraud Detection employs ABC-driven optimization to refine feature selection and capture complex fraud patterns that clustering may overlook. Using these optimized outputs, the KNN classifier performs instance-based classification by calculating the distance between the trained transaction and ABC-optimized reference points from training, selecting the k nearest neighbors, and applying a majority-vote mechanism to determine whether a transaction is fraudulent or legitimate. The KNN decision is further combined with intermediate results through decision-level fusion, ensuring a robust, accurate, and real-time fraud detection process [58].

3.2 Phase 2: Testing phase

During testing, the proposed model processes online transactions through a compact multi-stage classification pipeline for efficient and accurate fraud detection. Transaction attributes are first extracted and evaluated against predefined fraud rules for initial filtering. Suspicious transactions are then classified using a KNN algorithm, leveraging optimized reference points generated by the ABC algorithm during training. ABC optimization refines clustering and classification boundaries, enabling KNN to assign labels based on proximity measures. While training is computationally intensive, the testing phase is lightweight, ensuring fast, accurate, and reliable real-time fraud detection.

3.2.1 Step 1: Transaction feature extraction

When an online transaction request is initiated, the system extracts key attributes such as transaction amount, timestamp, location, merchant ID, payment method, and customer-specific behavioral patterns.

3.2.2 Step 2: Initial rule-based filtering

The extracted attributes are evaluated against a predefined set of fraud detection rules designed from historical behavioral profiles and business policies. Examples of rules include abnormal spending limits, mismatched shipping and billing addresses, rapid transaction bursts, and unusual geolocation access. Transactions that meet these anomaly criteria are flagged as suspicious and forwarded to the next stage for deeper evaluation. Transactions that fully comply with normal behavior proceed directly to approval, thereby reducing unnecessary computation.

3.2.3 Step 3 : ABC-optimized KNN classification

Transactions requiring further inspection are processed by the KNN classifier. During training, the ABC algorithm optimized the clustering boundaries and generated high-quality synthetic samples to handle class imbalance. In the testing phase, KNN uses these ABC-optimized reference points to classify the incoming transaction. A distance metric (Euclidean distance) is computed between the new transaction and stored reference points. The transaction is assigned a label — fraudulent or legitimate — based on the majority class of its k-nearest neighbors.

3.2.4 Step 4 — Decision fusion and final classification

In the proposed framework, the KNN classifier serves as the final decision-making stage by classifying transactions based on their similarity to ABC-optimized reference points generated during training. It calculates the distance between the incoming transaction and historical labeled samples, selects the k nearest neighbors, and applies a majority-vote mechanism to determine whether the transaction is fraudulent or legitimate. The KNN output is then combined with results from Rule-Based Filtering, K-means Clustering, and ABC-based Swarm Fraud Detection through decision-level fusion, ensuring a highly accurate, robust, and real-time fraud detection process.

3.2.4.1 DIDS framework To enable collaborative fraud mitigation in a DIDS framework, the proposed model integrates a robust inter-node communication mechanism designed for secure, real-time information sharing among participating nodes. Each node operates autonomously but contributes to a collective threat intelligence network by exchanging detected fraud patterns, flagged transactions, and behavioral anomalies. This is facilitated through secure communication channels using Transport Layer Security (TLS) for encryption, while Protocol Buffers are adopted for efficient, lightweight, and cross-platform message serialization. For distributed coordination, Apache Kafka is utilized to manage high-throughput, low-latency message streams between nodes, enabling near-instantaneous updates and synchronization across the network without overloading the system.

When a node identifies a suspicious pattern—such as abnormal transaction frequency, geolocation anomalies, or high-risk feature combinations—it generates a cryptographic signature of the detected pattern using the SHA-256 hashing algorithm. This signature is then broadcast to the network using a publish-subscribe model via Kafka topics or recorded on a shared distributed ledger powered by like Hyperledger Fabric framework. Utilizing distributed ledger technologies ensures data immutability, traceability, and non-repudiation, allowing all participating nodes to verify the authenticity and integrity of shared intelligence. Consequently, every node maintains a synchronized fraud knowledge base, enabling real-time cross-referencing of incoming transactions against a continually updated repository of known fraudulent profiles.

This cooperative filtering mechanism significantly enhances the scalability, efficiency, and reliability of the proposed DIDS framework by minimizing redundant

computations and accelerating fraud detection. The Elasticsearch engine is integrated to provide fast, distributed, and fault-tolerant storage for fraud signatures and flagged transactions, ensuring quick retrieval during real-time analysis. To support streaming data analytics, the model leverages Apache Flink, which enables low-latency event processing and efficient correlation of suspicious patterns across the network. Furthermore, Public Key Infrastructure (PKI) is adopted to ensure secure node authentication, digital signing, and encrypted inter-node communication, preventing unauthorized access and tampering. By integrating these distributed computing, storage, and security tools, the proposed DIDS model guarantees high availability, rapid response, and robust resilience against sophisticated and coordinated fraud attempts.

4 Experimental results

To validate the proposed model for detecting online fraudulent trading in FinTech, a series of experiments were conducted to evaluate its robustness, effectiveness, and generalization capabilities, particularly in addressing class imbalance, anomaly detection, and fraud classification. Performance was measured using precision, recall, and F-measure. Experiments were executed on a system with an Intel® Core™ i5 processor and 8 GB RAM using MATLAB R2015a. Class imbalance was mitigated using the ABC-sampling technique, which employs artificial bee colony-inspired optimization to generate synthetic samples while preserving critical discriminative features. Additionally, a synthetic dataset of 100,000 transactions, implemented in C#, was used to test the model's scalability and accuracy with large-scale data.

The dataset used in this study is sourced from Kaggle's "Online Payments Fraud Detection Dataset", <https://www.kaggle.com/datasets/rupakroy/online-payments-fraud-detection-dataset>, which contains anonymized transactional records representing online payment activities. The dataset includes 1,000 transactions involving 99 unique customers, with 927 transactions used for training and 100 for testing. It comprises 13 attributes: 12 input features and one binary class label indicating whether a transaction is fraudulent or legitimate. Although the dataset is anonymized, it simulates realistic e-commerce transaction patterns, including behavioral, temporal, and monetary attributes. For preprocessing, missing values were not present in the dataset, allowing the model to utilize all records without imputation. Extreme values, particularly in financial fields like transaction amount, were assessed for outliers using Interquartile Range (IQR)-based filtering to ensure that unusually high or low values did not skew the training process, unless they corresponded to potentially fraudulent activity—in which case, they were retained to preserve meaningful fraud signals. The data was also normalized to ensure uniform feature scaling, which is critical for many machine learning algorithms to perform effectively. This dataset includes temporal information in the form of a step attribute, where each step represents one hour from the start of data collection. This provides a relative timeline that allows for tracking the sequence and duration of transactions (e.g., a transaction at step=100 occurred 100 h into the dataset). However, the dataset does not include actual timestamps or calendar dates, meaning it lacks information about specific days, months, or years. As a result, while temporal patterns can be analyzed within the dataset's

own timeline, it is not possible to align transaction activity with real-world events, seasons, or holidays. Thus, the temporal aspect is limited to internal chronological order rather than absolute time references.

Kaggle’s “Online Payments Fraud Detection Dataset” contains a highly imbalanced distribution, with fraudulent transactions comprising less than 5% of the total data. To address this significant class imbalance and ensure the model is exposed to a diverse range of fraudulent patterns, synthetic data was generated using the suggested ABC-sampling technique. This method not only increased the representation of minority class samples but also preserved critical feature relationships, enhancing the model’s learning process. The synthetic data aimed to simulate various realistic fraud scenarios, including account takeovers, bursts of high-frequency micro-transactions, unusual transaction amounts, irregular timing patterns, and deviations in typical user behavior. By incorporating these simulated behaviors, the augmented dataset provides a more comprehensive and balanced foundation for training, allowing the model to better generalize and detect both known and emerging fraud strategies.

4.1 Experiment 1: Data preprocessing and class imbalance evaluation

This experiment evaluates the efficacy of the ABC-sampling technique in mitigating class imbalance for fraud detection, relative to conventional sampling strategies such as SMOTE, oversampling, and undersampling. Model performance is assessed using key metrics—including accuracy, precision, recall, and F-measure—to determine the effectiveness of each method in correctly identifying fraudulent transactions. The experimental procedure involves data preprocessing, the application of sampling methods, followed by model training and testing, with the performance outcomes presented in Table 3. Oversampling demonstrates moderate effectiveness (Accuracy: 82.5%, F-Measure: 67.7%) by replicating minority-class instances, which improves recall but introduces overfitting risks and limits representation of diverse fraudulent patterns. Undersampling enhances precision (74.1%) by reducing the majority class; however, it substantially lowers recall (53.4%) due to the removal of potentially informative legitimate transactions. SMOTE, which generates synthetic minority samples via interpolation, achieves improved performance across all metrics (Accuracy: 88.1%, F-Measure: 79.5%), offering a more balanced trade-off between precision and recall, although the generated synthetic data may introduce noise. The ABC-sampling approach outperforms all comparative techniques (Accuracy: 95.7%, F-Measure: 87.3%) by integrating feature importance analysis with optimization mechanisms inspired by artificial bee colony behavior to produce high-fidelity synthetic samples. This method preserves crucial discriminative features, thereby enhancing the reliability and accuracy of fraud detection.

Initially, PSO-Sampling used a population size of 30, inertia weight $w = 0.9$, and acceleration coefficients $c1 = c2 = 2.0$; through tuning, the final configuration adjusted w to 0.7 and improved the fitness (accuracy) from 86.4% to 93.6%. ACO-Sampling began with 20 ants, an evaporation rate of 0.5, and pheromone importance $\alpha = 1$; after optimization, it employed 30 ants, a reduced evaporation rate of 0.3, and enhanced pheromone influence, boosting the fitness value from 85.9% to 92.9%. The proposed ABC-Sampling started with 20 food sources, a limit of 100,

and a colony size of 40; final parameters were refined to 30 food sources and a limit of 150, increasing accuracy from 88.2% to 95.7%. These improvements underscore the importance of hyperparameter tuning in optimizing synthetic sample quality and achieving superior anomaly detection performance. The main configuration for Oversampling and Undersampling involves adjusting the class distribution ratio, typically set to achieve a balanced dataset (e.g., 1:1) between majority and minority classes without generating or removing excessive data. For SMOTE, key parameters include the number of nearest neighbors (commonly set to $k = 5$) and the oversampling percentage, which controls how many synthetic samples are generated based on feature-space interpolation between existing minority instances.

The results presented in Table 3 clearly demonstrate that optimization-based sampling techniques—namely PSO-Sampling, ACO-Sampling, and ABC-Sampling—significantly outperform traditional sampling methods such as Oversampling, Undersampling, and SMOTE. Among the optimization methods, PSO-Sampling and ACO-Sampling show strong performance improvements, achieving F-measures of 85.1% and 84.4%, respectively. These results indicate that the use of metaheuristic algorithms for sample selection helps generate more representative and balanced datasets, thereby enhancing the ability of the fraud detection model to generalize and detect minority class instances more effectively. This improvement is primarily due to the intelligent exploration and exploitation capabilities inherent in these algorithms, which allow them to focus on the most informative samples.

Despite the competitive performance of PSO and ACO, ABC-based sampling technique achieves the highest performance across all evaluation metrics due to its superior ability to balance exploration (searching new areas of the solution space) and exploitation (refining known good solutions). ABC simulates the natural behavior of a bee colony, where three types of agents—employed bees, onlooker bees, and scout bees—work in coordination to explore and exploit food sources (solutions). Employed bees search around known good solutions, onlooker bees probabilistically select promising areas based on quality, and scout bees introduce randomness by exploring completely new areas. This structure naturally divides responsibilities among agents, allowing the algorithm to dynamically shift between broad search and fine-tuning based on the current search status. In contrast, PSO tends to converge quickly due to particles being pulled toward global and personal bests, which can lead to premature convergence. ACO, while effective in path-finding problems, relies heavily on pheromone updates and suffers from stagnation if diversity is not well maintained. ABC's decentralized and probabilistic exploration mechanisms help

Table 3 Performance comparison of sampling techniques for fraud detection

Sampling Technique	Accuracy	Precision	Recall	F-Measure
Oversampling	82.5%	70.4%	65.2%	67.7%
Undersampling	76.3%	74.1%	53.4%	62.1%
SMOTE	88.1%	81.7%	77.4%	79.5%
PSO-Sampling	93.6%	87.1%	83.2%	85.1%
ACO-Sampling	92.9%	86.4%	82.5%	84.4%
ABC-Sampling (Proposed Model)	95.7%	89.2%	85.6%	87.3%

avoid such pitfalls, making it better suited for diverse sample space exploration like in imbalanced data scenarios.

In addition, ABC's dynamic search mechanism distinguishes itself from other optimization techniques by its adaptive local search strategy. Each employed bee modifies a candidate solution by applying a stochastic update based on the difference between the current solution and a randomly chosen neighbor. This differential-based update introduces self-adaptive perturbations, which are automatically scaled based on the diversity of the population. If the population is diverse, the updates encourage exploration; if it becomes more homogenous, the updates become smaller, supporting exploitation. Moreover, the scout bee mechanism serves as an implicit restart strategy, replacing stagnant solutions with entirely new random ones if no improvement is found over several cycles. This prevents the algorithm from getting trapped in local optima, a common limitation in PSO and ACO, where convergence behavior is often dictated by fixed parameters or accumulated pheromones. As a result, ABC dynamically and autonomously adjusts its search behavior in response to the search space landscape, making it highly resilient, self-tuning, and effective for complex sampling tasks like fraud detection under severe class imbalance.

4.2 Experiment 2: Feature importance and selection analysis

The analysis of feature importance within the ABC-sampling algorithm demonstrates that certain attributes play a critical role in generating high-quality synthetic samples for fraud detection. As shown in Table 4, "Amount of Transaction" and "Location of Transaction" exhibit the highest importance scores (0.35 and 0.32) with strong correlations to classification outcomes (+0.88 and +0.85), indicating their pivotal influence on accurate fraud identification. Features such as "Current Balance" (0.28) and "Card Used" (0.25) also significantly contribute, while lower-importance attributes, including "Number of Overdrafts" (0.10) and "Location of Card Used" (0.12), exert minimal impact on both sample quality and detection performance. Moderately influential features like "Average Bank Balance," "Credit Card Age," and "Authentication Type" play a supporting role in refining synthetic samples. These results suggest that emphasizing high-importance features enhances the fidelity of synthetic data and improves overall fraud detection accuracy.

To further solidify the model interpretation, statistical hypothesis testing (t-tests, and ANOVA) is conducted to validate whether differences in feature distributions between fraudulent and legitimate transactions are statistically significant. These tests help ensure that the identified correlations are not due to random chance, thus reinforcing confidence in the model's decisions. For example, if Authentication Type shows a statistically significant difference between fraud and non-fraud cases, even with a moderate importance score (+0.72 correlation), it adds interpretive value. Such analysis makes the model not just more accurate but also more transparent and explainable, a critical requirement in financial domains.

5 Independent samples t-test (for continuous features)

Purpose: To compare the means of two groups (fraudulent vs. legitimate transactions) for each continuous variable to determine if they are significantly different.

Null hypothesis (H_0) *The mean of the feature is the same for fraudulent and legitimate transactions.*

Alternative hypothesis (H_1) *The means are significantly different. The results are presented in Table 5, along with their corresponding justifications.*

6 One-way ANOVA (for categorical features with more than two groups)

Purpose: To examine if there are statistically significant differences in fraud rates across multiple groups within a categorical feature (see Table 6).

Authentication Type (e.g., 'PIN', 'OTP', 'Biometric', 'None').

Null hypothesis (H_0) *Fraud rates are equal across all authentication types.*

Alternative hypothesis (H_1) *At least one group has a different fraud rate.*

6.1 Experiment 3: Behavioral profile creation and anomaly detection

The primary aim of this experiment is to evaluate the efficacy of a K-Means clustering-based Level-1 classification framework in constructing behavioral profiles for legitimate and fraudulent traders. Utilizing historical transaction data, the system seeks to uncover intrinsic patterns that facilitate accurate clustering into these two categories, establishing a baseline for normative behavior and anomaly detection. The experiment quantifies the model's effectiveness in fraud detection by minimizing

Table 4 Feature importance analysis and its impact on synthetic sample quality for fraud detection

Feature	Importance score	Correlation with classification outcomes	Impact on synthetic sample quality
Location of transaction	0.32	+0.85	High
Current balance	0.28	+0.80	High
Average bank balance	0.22	+0.76	Moderate
Amount of transaction	0.35	+0.88	High
Credit card age	0.15	+0.70	Moderate
Authentication type	0.18	+0.72	Moderate
Location of card used	0.12	+0.68	Low
Number of overdraft	0.10	+0.65	Low
Overdraft transaction	0.20	+0.74	Moderate
Card used	0.25	+0.78	High
Card used transaction	0.17	+0.71	Moderate

misclassification and overlap between the groups, thereby ensuring robustness and practical applicability in real-world trading environments. This foundational evaluation also highlights the model's strengths and limitations, providing insights for further optimization of clustering strategies and feature engineering in subsequent classification stages. As shown in Table 7, the k-means clustering model achieves an overall accuracy of 91.6% in behavioral profile classification, effectively distinguishing between legitimate and fraudulent traders. Legitimate traders are classified with 92.6% accuracy and 92% precision, while fraudulent traders achieve 90.2% accuracy and 85% precision, indicating strong detection capability but a moderate false-positive rate for anomalous behaviors. Recall values of 88% for legitimate and 87% for fraudulent traders, combined with an overall F1-score of 89.5%, reflect a well-balanced trade-off between precision and recall. These findings demonstrate that k-means clustering provides a robust foundation for initial-level behavioral profiling, effectively capturing both normative and anomalous trading patterns, while highlighting potential improvements in fraudulent detection precision through hierarchical or multi-level classification.

6.2 Experiments 4: Performance benchmarking

The proposed fraud detection algorithm is benchmarked against traditional machine learning models, deep learning architectures, and statistical anomaly detection methods, with comparative evaluation across accuracy, precision, recall, F1-score, AUC-ROC, and computational efficiency to demonstrate its superior effectiveness and robustness in detecting fraudulent transaction. The comparative models were configured using standard yet optimized hyperparameters suitable for fraud detection tasks. Random Forest was implemented with 100 trees and a maximum depth of 10, balancing interpretability and performance. Support Vector Machines used an RBF kernel with $C = 1.0$ and $\gamma = 0.01$, targeting non-linear separation. Logistic Regression applied L2 regularization with a regularization strength $C = 1.0$. Neural Networks used a three-layer architecture with ReLU activations and a learning rate of 0.001, while LSTM models comprised two hidden layers with 64 and 32 units respectively, optimized using the Adam optimizer. GBM employed 200 estimators, a learning rate of 0.05, and a maximum depth of 6. For unsupervised methods, Isolation Forest was run with 100 estimators and contamination set to 0.1, while Clustering-Based Outlier Detection used k-means with $k = 5$. Z-Score analysis and other statistical methods relied on standard thresholding (typically $\pm 3\sigma$) to flag anomalies. These configurations ensured a fair and consistent comparison across models. The proposed model utilizes the same hyperparameter configuration as specified in Experiment 1.

The comparative analysis presented in Table 8 clearly demonstrates the superiority of the proposed algorithm over prominent deep learning and ensemble learning approaches such as Neural Networks, LSTM, and GBM. The proposed model outperforms these techniques across all key evaluation metrics, achieving the highest accuracy (95.0%), F1-Score (92.7%), and AUC-ROC (0.97), while also maintaining real-time computational efficiency (120 ms). In contrast, Neural Networks, although robust (90.0% accuracy, 0.92 AUC-ROC), exhibit slower response times (250 ms), making them less suited for time-sensitive fraud detection systems. Similarly, while

Table 5 Independent samples t-test results

Feature	Mean (Fraud)	Mean (Legit)	<i>p</i> -value	Significance ($p < 0.05$)	Interpretation
Location of transaction	6.3	3.1	0.0002	✓ Significant	Strong geographical variance in fraud cases
Current balance	1200.5	2400.3	0.015	✓ Significant	Lower balances linked to fraud
Average bank balance	1100.8	1300.2	0.092	✗ Not Significant	Mean difference not statistically relevant
Amount of transaction	785.2	245.1	0.0001	✓ Significant	Fraudulent transactions often have higher amounts
Credit card age (years)	1.2	3.6	0.003	✓ Significant	Newer cards are more often targeted
Authentication type*	N/A (categorical)	N/A	—	ANOVA	Use ANOVA for categorical variables
Location of card used	5.2	3.4	0.048	✓ Significant	Location mismatch may trigger suspicion
Number of overdraft	2.1	1.2	0.068	✗ Not Significant	Small difference; not reliable alone
Overdraft transaction	1.8	0.9	0.027	✓ Significant	Fraud linked to overdraft behavior
Card used	3.7	2.4	0.009	✓ Significant	Multiple card usage may indicate fraud
Card used transaction	2.8	1.5	0.021	✓ Significant	Frequency of card transactions matters

LSTM models excel in modeling temporal dependencies with commendable performance (91.5% accuracy, 0.94 AUC-ROC), their computational cost (300 ms) makes them impractical for real-time deployment in high-throughput environments such as online financial transactions. GBM, a powerful ensemble method, also shows high performance (92.5% accuracy, 0.95 AUC-ROC), yet it falls short of the proposed model in both effectiveness and speed (200 ms), and is further limited by its tendency to overfit and its growing complexity as data size increases.

Table 6 One-Way ANOVA analysis for fraud rate distribution across different authentication methods (ANOVA p-value = 0.0004)

Authentication Type	Fraud Rate (%)
PIN	1.2%
One-Time Password(OTP)	3.5%
Biometric	0.5%
None	6.8%

The p-value < 0.05 indicates that fraud occurrence significantly varies depending on the authentication method. Particularly, ‘None’ and ‘OTP’ types are more susceptible to fraud, adding explanatory power to this feature. Despite its moderate correlation (+ 0.72), its statistical significance validates its inclusion in the model

Table 7 Performance metrics for behavioral profile classification using k-means clustering (level 1 classification)

Cluster	Metric	Value
Legitimate	Accuracy	92.6%
Fraudulent	Accuracy	90.2%
Total	Accuracy	91.6%
Legitimate	Precision	92%
Fraudulent	Precision	85%
Legitimate	Recall	88%
Fraudulent	Recall	87%
Total	F1 Score	89.5%

The superior performance of the proposed model can be attributed to its optimized architecture, which likely integrates advanced mechanisms for capturing both non-linear and sequential patterns while maintaining computational efficiency. Unlike standard Neural Networks that lack memory capabilities, or LSTM models that require heavy resources to model long-term dependencies, the proposed approach seems to strike an effective balance between learning capacity and operational speed. Moreover, GBM, though highly accurate in many tabular data tasks, is inherently sequential in its boosting process, making it slower and harder to parallelize, which impacts scalability and latency. Additionally, GBM and LSTM models are more susceptible to overfitting when exposed to high-dimensional or imbalanced data, a common issue in fraud detection. In contrast, the proposed algorithm appears to mitigate these drawbacks, delivering high generalization performance with minimal latency, making it an ideal choice for real-world, real-time fraud detection systems.

6.3 Experiment 5: Impact of data size and quality

Fraud detection systems in real-world settings often face datasets of varying size and quality, including noisy or incomplete records. To evaluate the robustness of the proposed model, its performance was tested under reduced dataset sizes (50%, 75%, 100% of the original data) and degraded data quality, including noise, mislabeled entries, and missing values. Metrics such as accuracy, precision, recall, F1-score, and AUC-ROC were monitored to assess performance sensitivity. The model’s ability to handle incomplete data was evaluated using imputation strategies, including mean, median, and k-nearest neighbors (KNN) imputation, to estimate missing values and

preserve the integrity of critical feature. Table 9 illustrates that reducing the dataset size affects the model's performance metrics, though the system maintains reasonable robustness. With 50% of the original data, accuracy decreases from 95.0% to 85.0%, indicating a modest reduction in the model's overall correctness in classifying transactions. Precision declines from 92.0% to 82.0%, suggesting that the proportion of correctly identified fraudulent transactions relative to all predicted frauds drops slightly with less training data. Recall decreases from 93.5% to 84.0%, reflecting a lower ability to detect all actual fraudulent transactions in smaller datasets. The F1-score, which balances precision and recall, falls from 92.7% to 83.0%, indicating that the model's overall predictive reliability diminishes with limited data. The AUC-ROC drops from 0.97 to 0.89, highlighting a slight reduction in the model's discriminative ability between fraudulent and legitimate transactions. These changes collectively demonstrate that while performance declines with reduced data, the model maintains sufficient robustness to generalize even with smaller datasets.

Introducing noise into the dataset also impacts model performance. With 5% noise, accuracy drops to 88.0%, indicating a slight increase in misclassifications due to corrupted or mislabeled data. Precision decreases to 84.0%, suggesting that some predicted frauds are incorrectly flagged, while recall rises slightly to 86.5%, showing the model still captures a majority of actual fraudulent transactions. The F1-score reduces to 85.0%, reflecting a balance between precision and recall under noisy conditions. The AUC-ROC at 0.91 shows the model remains relatively effective at distinguishing fraud from legitimate transactions. At 10% noise, accuracy and all other metrics decline further (accuracy 82.0%, precision 79.5%, recall 81.0%, F1-score 80.2%, AUC 0.86), indicating that higher levels of data corruption challenge the model's reliability. These results highlight the model's sensitivity to noise and underscore the importance of preprocessing and noise-handling techniques to maintain performance.

Table 8 Comparison of fraud detection methods across key performance metrics

Model/Method	Accuracy (%)	Precision (%)	Recall (%)	F1-Score (%)	AUC-ROC	Computational Efficiency (ms)
Proposed Algorithm	95.0	92.0	93.5	92.7	0.97	120
Random forest	88.0	85.0	86.0	85.5	0.89	150
Support vector machines	85.0	83.5	82.0	82.7	0.86	180
Logistic Regression	80.0	77.0	75.0	76.0	0.81	100
Neural Networks	90.0	88.0	87.5	87.7	0.92	250
Deep learning long short-term memory (LSTM)	91.5	89.0	88.5	88.7	0.94	300
Ensemble learning Gradient boosting machine (GBM)	92.5	90.0	89.5	89.7	0.95	200
Z-score analysis	75.0	70.0	72.0	71.0	0.75	80
Isolation forest	83.0	78.5	80.0	79.2	0.84	100
Clustering-based outlier detection	78.0	74.0	73.0	73.5	0.78	90

The model demonstrates resilience to missing data, though performance is affected as the proportion of incomplete records increases. With 5% missing values, accuracy is 90.5%, precision 87.0%, recall 89.0%, F1-score 88.0%, and AUC 0.92, indicating minimal impact due to effective handling through imputation techniques. This suggests that small gaps in critical features do not significantly compromise fraud detection. However, with 10% incomplete records, accuracy drops to 85.5%, precision to 83.0%, recall to 84.5%, F1-score to 83.7%, and AUC to 0.88, showing a more pronounced reduction in model effectiveness. These observations highlight the importance of maintaining feature completeness and employing robust imputation strategies for critical variables. Overall, the table demonstrates that while dataset quality and completeness influence performance, the model exhibits strong resilience, maintaining reasonable predictive capabilities even under challenging data conditions.

6.4 Experiment 6: Scalability and real-time performance

This experiment aims to evaluate the algorithm's performance in real-world trading environments, where rapid and precise decision-making is essential. The experiment assesses key metrics such as latency, throughput, and real-time detection accuracy to determine the system's scalability and operational efficiency under varying transaction loads. Table 10 presents the performance evaluation of the real-time fraudulent trading detection system under varying transaction volumes and fraud prevalence rates defined as the proportion of fraudulent transactions within the total transaction stream, reflects the system's exposure to detection challenges. As transaction throughput increases from 500 to 10,000 transactions per second and fraud prevalence rises from 1% to 30%, the system demonstrates a gradual increase in average latency, from 50 ms to 180 ms, reflecting the additional computational overhead associated with higher data volumes and more complex detection scenarios. Despite this increase, throughput remains consistently close to the incoming transaction rate, indicating that the system maintains strong real-time processing capability and scalability. Concurrently, real-time detection accuracy exhibits a modest decline, decreasing from 98.5% to 92.3%, which can be attributed to the heightened difficulty of distinguishing fraudulent transactions in larger and noisier data streams. Overall, these results indi-

Table 9 Impact of data-set size and quality on model performance

Data Condition	Accuracy (%)	Precision (%)	Recall (%)	F1-Score (%)	AUC-ROC
Original Dataset	95.0	92.0	93.5	92.7	0.97
50% Dataset	85.0	82.0	84.0	83.0	0.89
75% Dataset	90.0	88.0	89.5	88.7	0.93
Noisy Data (5%)	88.0	84.0	86.5	85.0	0.91
Noisy Data (10%)	82.0	79.5	81.0	80.2	0.86
Incomplete Records (5%)	90.5	87.0	89.0	88.0	0.92
Incomplete Records (10%)	85.5	83.0	84.5	83.7	0.88

cate that the system is capable of efficiently handling high-volume, high-prevalence trading environments, though further optimization may be required to preserve peak detection accuracy under extreme operational condition.

6.5 Experiment 7: Sensitivity analysis

Hyperparameter tuning is critical in machine learning models using ABC-sampling to ensure stable and efficient performance. This experiment evaluates the impact of the sampling rate and pheromone evaporation rate on key metrics including accuracy, precision, recall, and F1 score. By identifying optimal parameter settings, the model can achieve improved efficiency while maintaining robustness across diverse scenarios. The findings provide practical guidance for reliable performance under varying configurations. In this experiment, the proposed ABC-Sampling algorithm was initially configured with 20 food sources, a colony size of 40, and a limit value of 100, serving as a balanced baseline for exploration and exploitation. After fine-tuning, the configuration was optimized to 30 food sources and an increased limit of 150, enabling deeper exploration of the solution space and reducing premature abandonment of promising solutions.

The results shown in Table 11 clearly demonstrate that both the sampling rate and the limit value in the ABC algorithm play a critical role in enhancing anomaly detection performance. As the sampling rate increases from 30% to 100%, overall accuracy, precision, recall, and F1 score steadily improve, indicating that higher sampling rates allow the model to better capture the underlying patterns in minority class data by generating a more diverse and representative set of synthetic samples. The limit value, which defines the maximum number of unsuccessful attempts before abandoning a solution, controls the balance between exploration and exploitation. At lower sampling rates (e.g., 30%), increasing the limit value from 50 to 100 improves performance, but further increase to 150 leads to degradation, likely due to overfitting on insufficient sample diversity. However, at higher sampling rates (e.g., 100%), a limit value of 100 yields the best performance (93.7% accuracy), suggesting that with sufficient data coverage, moderate exploration is optimal. These findings highlight the importance of dynamically adjusting the limit value in accordance with the sampling rate to maintain algorithmic stability and avoid premature convergence or excessive stagnation, thereby ensuring robust anomaly detection performance.

Furthermore, the results in Table 11 reveals a clear trend indicating that increasing the number of food sources positively impacts model performance across all evalu-

Table 10 Performance metrics for Real-Time fraudulent trading detection system

Test ID	Data volume (transactions/second)	Fraud prevalence (% of total transactions)	Average Latency (ms)	Throughput (transactions/second)	Real-Time Accuracy (%)
1	500	1%	50	495	98.5
2	1000	5%	75	980	97.2
3	2500	10%	90	2,450	95.8
4	5000	20%	120	4,850	94.5
5	10,000	30%	180	9,700	92.3

ation metrics—accuracy, precision, recall, and F1 score. At fixed sampling rates and limit values, higher food source counts consistently yield better results. For instance, at a sampling rate of 100% and a limit value of 100, accuracy improves from 93.7% to 94.5% as food sources increase from 20 to 60. This pattern suggests that more food sources enable broader exploration of the solution space by the ABC algorithm, enhancing its ability to discover more optimal and diverse sampling strategies. As a result, the model becomes better at capturing informative patterns, particularly for the minority class, leading to improved detection of fraudulent instances. However, the performance gains tend to stabilize at higher food source values, indicating a point of diminishing returns where additional food sources contribute less significantly to further improvement.

6.6 Experiment 8: Cross-dataset evaluation of FinTech anomalies

To validate the robustness and generalizability of the proposed ABC-based anomaly detection framework, we designed an extended experiment that evaluates the system across multiple datasets for each of the four major fraud-related anomaly types in the FinTech domain: transactional fraud, money laundering, market manipulation, and operational fraud. The objective is to test whether the model maintains high performance and adaptability across various real-world financial fraud scenarios, despite differing data structures, volumes, and behavioral characteristics. The experimental configuration involves using a dual-dataset setup for each fraud type, applying consistent preprocessing steps such as feature normalization and class balancing using the proposed ABC-based sampling strategy. For transactional fraud, we utilize the Credit Card Fraud Detection Dataset from Kaggle (<https://www.kaggle.com/dataset/mlg-ulb/creditcardfraud>), containing 284,807 transactions with highly imbalanced labels, and the IEEE-CIS Fraud Detection Dataset (<https://www.kaggle.com/competitions/ieee-fraud-detection>), which includes anonymized e-commerce transaction records enriched with device and identity-related features.

For money laundering, the experiment uses the Elliptic Bitcoin Dataset (<https://www.kaggle.com/datasets/ellipticco/elliptic-data-set>), which comprises over 200,000

Table 11 Performance metrics for varying sampling Rates, limit value Rates, and food source

Sampling Rate (%)	Limit Value	Food Sources	Accuracy (%)	Precision (%)	Recall (%)	F1 Score (%)
30	50	20	80.5	78.2	77.8	78.0
30	50	40	81.2	79.1	78.3	78.7
30	50	60	82.0	80.0	79.2	79.6
30	100	20	82.3	80.1	79.9	80.0
30	100	40	83.0	81.0	80.4	80.7
30	100	60	83.8	81.9	81.1	81.5
70	100	20	86.5	84.2	83.9	84.0
70	100	40	87.1	85.0	84.4	84.7
70	100	60	87.8	85.7	85.1	85.4
100	100	20	93.7	91.3	91.0	91.2
100	100	40	94.1	91.9	91.5	91.7
100	100	60	94.5	92.4	92.0	92.2

Bitcoin transactions labeled as licit, illicit, or unknown, and the AMLSim Dataset, a simulated dataset provided by the Financial Conduct Authority (FCA), designed to mimic suspicious bank transfers and laundering activities across financial entities. For market manipulation, we include the LOBSTER dataset, which provides high-frequency limit order book data from the Frankfurt Stock Exchange—suitable for detecting spoofing and layering—and the DEBS 2019 Challenge dataset, simulating real-time trading and insider activities. For operational fraud, the SWaT dataset (http://itrust.sutd.edu.sg/itrust-labs_datasets/dataset_info/) offers ICS sensor data labeled with cyberattacks, and the Numenta Anomaly Benchmark (NAB) (<https://numenta.org/nab/>), which contains real-time financial and system metrics with contextual anomalies. Performance is measured using key evaluation metrics including Accuracy, Precision, Recall, F1-score, and AUC-ROC. This comprehensive setup ensures a rigorous cross-context validation of the proposed model's ability to detect a wide spectrum of FinTech fraud types in both centralized and distributed environments.

Adapting the proposed ABC-based anomaly detection framework—initially designed for transactional fraud—to other types of FinTech anomalies such as money laundering, market manipulation, and operational fraud necessitates a strategic reconfiguration of feature extraction and standardization. While transactional fraud utilizes well-structured features like transaction amount, timestamps, merchant ID, and user behavior, extending the model to other fraud types requires incorporating domain-specific attributes. For example, money laundering detection benefits from graph-based features such as transaction paths, account relationships, and origin-destination flows, while market manipulation analysis requires high-frequency trading metrics like order book depth, price volatility, and order cancellation rates. Operational fraud detection, on the other hand, relies heavily on time-series sensor data and system logs, where features such as login attempts, error codes, and CPU/memory usage indicate anomalous patterns. Each fraud type necessitates tailored behavioral profiling—whether of users, traders, or devices—using appropriate clustering or sequence modeling techniques. Herein, we implement a unified meta-feature schema that maps domain-specific features (e.g., transactional, graph-based, time-series) into common categories like temporal, behavioral, topological, and system-level metrics, followed by normalization and embedding (e.g., PCA) to form consistent input vectors. This allows the ABC-based model to process heterogeneous FinTech datasets within a standardized, comparable feature space for effective anomaly detection.

The results shown in Table 12 demonstrate that the proposed ABC-based anomaly detection model consistently delivers high performance across all four categories of FinTech fraud, regardless of dataset type or underlying data structure. Accuracy values range between 92.1% and 96.2%, with AUC-ROC scores consistently above 0.93, indicating that the model maintains strong discriminatory power even when dealing with imbalanced or noisy datasets. This consistency is especially important in high-stakes environments like financial systems, where the cost of undetected fraud is significant. The model's ability to sustain high recall scores (up to 89.7%) while maintaining balanced precision reflects its effectiveness in identifying fraudulent activities without overwhelming false positives.

The minor fluctuations across datasets—for example, slightly lower recall on the NAB dataset (82.3%) and LOBSTER (83.2%)—can be attributed to the inherent

variability in anomaly density and complexity. In LOBSTER, the high-frequency trading data is more volatile, and in NAB, contextual anomalies may not follow easily generalizable temporal patterns. Despite these challenges, the ABC-based model adapts effectively, due in large part to its swarm intelligence-driven sampling, which improves minority class representation during training without introducing synthetic noise. The consistent F1-scores (above 83% in all cases) reflect the model's ability to generalize effectively and maintain balanced sensitivity and specificity across diverse fraud scenarios.

The superiority of the proposed ABC-based sampling technique becomes particularly evident when compared to traditional oversampling methods or even other metaheuristic sampling strategies like PSO and ACO. By focusing on the most informative features and dynamically exploring the sample space, the ABC algorithm creates more representative synthetic minority samples. This results in improved learning of subtle fraud patterns that might otherwise be overlooked. Moreover, integrating the model into a Distributed Intrusion Detection System (DIDS) further enhances its real-time detection and scalability, especially for operational fraud and system-level anomalies. The combination of intelligent sampling, behavioral modeling, and distributed monitoring makes the proposed framework a comprehensive and superior solution for robust FinTech fraud detection.

6.7 Experiment 9: Execution time analysis

The objective of last set of experiments is to evaluate the computational efficiency and scalability of each component in the proposed multi-stage model—ABC Sampling, Rule-based Classification, and Swarm-based K-means Classification—across varying dataset sizes. The experiment was configured by incrementally increasing the dataset size from 500 to 100,000 samples and measuring the execution time of each stage separately, as well as the total runtime. The model was executed on a standard hardware setup consisting of an Intel i5 processor with 8 GB of RAM, representing a typical research-level environment. The primary evaluation metric used was runtime in seconds, allowing assessment of the computational overhead associated with each module and its impact on overall system performance.

Table 12 Performance evaluation of the proposed ABC-Based anomaly detection framework across diverse fintech fraud types and datasets

Anomaly Type	Dataset	Accuracy (%)	Precision (%)	Recall (%)	F1-Score (%)	AUC-ROC
Transactional Fraud	Credit Card Fraud Dataset	96.2	91.5	89.7	90.6	0.974
	IEEE-CIS Fraud Detection	94.8	89.2	85.3	87.2	0.961
Money Laundering	Elliptic Bitcoin Dataset	93.6	88.1	84.9	86.5	0.957
	AMLSim Dataset	95.0	90.3	86.4	88.3	0.968
Market Manipulation	LOBSTER Dataset	92.4	86.7	83.2	84.9	0.944
Operational Fraud	DEBS 2019 Challenge Dataset	93.3	87.4	84.7	86.0	0.951
	SWaT Dataset	94.2	89.0	85.8	87.4	0.960
	NAB Dataset	92.1	85.6	82.3	83.9	0.938

Table 13 presents the runtime performance of each component in the proposed model—ABC Sampling, Rule-based Classification, and the Swarm-based K-means Classifier—across different dataset sizes ranging from 500 to 100,000 samples. As expected, the total runtime increases with the dataset size due to the greater computational demand for processing and optimization. ABC Sampling shows a near-linear growth trend, as it iteratively explores feature combinations to enhance selection quality. The rule-based classifier remains relatively lightweight and grows modestly, as its logic-driven decisions are executed efficiently even at larger scales. The most computationally intensive stage is the Swarm+K-means classification layer, which combines population-based search (swarm intelligence) with iterative clustering, resulting in nonlinear growth due to increased iterations and convergence complexity. Overall, the system remains computationally manageable for small to medium-sized datasets, but optimization strategies (e.g., parallelism or algorithmic refinement) may be needed for scalability in high-volume data environments.

In practical applications, managing the trade-off between model performance and computational cost involves balancing accuracy, scalability, and runtime efficiency to suit the specific context and resource availability. Based on the obtained results in the execution time table, we observe that as the dataset size increases, the total runtime rises significantly, especially due to the Swarm+K-means classification stage, which dominates computational cost at larger scales. However, this stage is critical for enhancing model accuracy by performing refined classification through clustering and optimization. To manage this trade-off effectively, several strategies are typically adopted:

- **Stage-wise optimization** From the results, the rule-based classifier contributes the least to runtime yet adds decision transparency and baseline classification efficiency. This suggests it's useful for quick filtering before engaging costlier stages. ABC sampling, though moderate in cost, helps reduce feature dimensionality, indirectly reducing downstream computational burden.
- **Adaptive component execution** In many real-world deployments, only a subset of the full pipeline is activated based on data volume or time constraints. For instance, for datasets under 2,000 samples (as shown, total time remains < 15 s), running the full model may be justifiable. For larger datasets (e.g., 10,000 samples with ~78 s runtime), selective component execution or reduced iteration settings in swarm optimization can significantly cut cost with minimal performance degradation.
- **Hardware-aware configuration** The model was tested on a mid-tier Intel i5 processor with 8 GB RAM, a typical setup for research or small-scale applications. In enterprise-level scenarios, deploying the same model on more powerful hardware (e.g., GPUs or parallelized CPU clusters).

6.8 System interpretability

The proposed model's multi-stage architecture—comprising ABC-based sampling, rule-based classification, and a swarm-based clustering layer—offers varying levels of interpretability across its components. The rule-based classifier provides the

Table 13 Execution time analysis of the proposed model components across varying dataset sizes

Dataset Size (Samples)	ABC Sampling (s)	Rule-based Classifier (s)	Swarm Classifier (s)	Total Run-time (s)
500	1.24	0.83	1.57	3.64
1,000	2.75	1.41	2.89	7.05
2,000	5.93	2.78	6.12	14.83
5,000	14.85	6.92	15.46	37.23
10,000	31.54	14.20	32.77	78.51
20,000	67.88	29.60	69.13	166.61
50,000	178.34	76.10	190.92	445.36
100,000	375.45	157.60	403.85	936.90

highest transparency, as its decisions are grounded in clearly defined logical rules based on expert insights or observed patterns, enabling straightforward traceability. In contrast, the ABC optimization classifier add complexity due to their heuristic and iterative nature, which can obscure decision rationales. To address this, techniques such as analyzing feature importance scores from the ABC stage can reveal which transaction attributes (e.g., amount, timing, frequency) are most influential in fraud detection, aligning the model's focus with domain knowledge. Visualization of cluster centroids and decision boundaries in the Swarm layer can help illustrate how the model separates fraudulent from legitimate behaviors, while labeling clusters with specific fraud types enhances interpretability by connecting learned patterns to known scenarios. Moreover, linking frequently selected features to observable behavioral anomalies—such as rapid transaction bursts or irregular amounts—provides intuitive, actionable insights. Together, these strategies bridge the gap between model complexity and interpretability, making the system more transparent, explainable, and trustworthy for real-world deployment.

6.9 System limitations

The system demonstrates strong potential in detecting online trading fraud, but several limitations remain. Its reliance on historical trading data to build behavioral profiles may hinder its ability to identify emerging or sophisticated fraud tactics, particularly those that evolve rapidly or imitate legitimate trading patterns. While ABC-sampling effectively addresses class imbalance, it introduces significant computational overhead, especially for large-scale, high-dimensional datasets, which could affect real-time applicability and scalability. The model's performance is highly dependent on the quality, diversity, and completeness of the training data; biases, missing values, or inconsistencies can compromise its ability to generalize to unseen or atypical fraudulent behaviors. Additionally, the clustering-based analysis assumes that fraudulent transactions form distinct, separable groups, which may not hold in scenarios where fraudsters deliberately blend their activities with normal trading patterns. Finally, although the system has shown improved accuracy over existing methods, its robustness across varying market conditions, financial instruments, and geographic regions remains untested, leaving open questions about its adaptability and reliability in real-world, heterogeneous environments.

7 Conclusions

The proposed model enhances fraud detection in fintech by combining ABC-sampling with a multi-level classification framework. ABC-sampling utilizes the collective intelligence of artificial bee colonies to generate synthetic samples based on feature importance, effectively balancing imbalanced datasets while retaining critical patterns. The multi-level classification framework operates in hierarchical stages: the first level applies anomaly detection to filter out highly irregular transactions, while the second level uses clustering algorithms to group trader behaviors and identify subtle deviations from normal patterns. This layered approach improves detection precision by progressively refining suspicious cases and minimizing false positives. Through behavioral profiling, the model detects anomalies such as unusual trading volumes, abnormal transaction timings, and inconsistent activity patterns. The suggested approach is scalable, adaptable to changing fraud strategies, and preserves essential feature integrity, ensuring hidden fraudulent activities are accurately captured. The modular design also allows seamless integration of additional detection techniques, enhancing flexibility for real-world fintech applications. Experimental results demonstrate a 10% improvement in detection accuracy compared to state-of-the-art methods, highlighting the model's robustness and effectiveness.

Future work will focus on improving the model's efficiency, adaptability, and generalization for large-scale and dynamic datasets. The ABC algorithm can be optimized through hybrid metaheuristics and parameter tuning to enhance convergence and handle imbalanced data. Techniques like transfer learning, multi-task learning, and online learning can improve performance across diverse markets and evolving fraud patterns. Additionally, integrating semi-supervised anomaly detection, unsupervised representation learning, and graph-based behavioral profiling will help uncover hidden relationships and detect emerging fraudulent activities in real time.

Author contributions Conceptualization, S.M.D., N.A.E., and A.I.S.; methodology, S.M.D., N.A.E. and A.A.E.; software, A.I.S.; validation, S.M.D., N.A.E. and A.A.E.; formal analysis, S.M.D. and N.A.E., and A.I.S.; investigation, S.M.D.; A.I.S. and N.A.E.; resources, A.I.S.; data curation, A.I.S.; writing—original draft preparation, S.M.D., and N.A.E., and A.I.S.; writing—review and editing, A.A.E. and A.I.S.; visualization, N.A.E. and A.I.S.; supervision, S.M.D.; project administration, A.I.S.; funding acquisition, A.I.S. All authors have read and agreed to the published version of the manuscript.

Funding Open access funding provided by The Science, Technology & Innovation Funding Authority (STDF) in cooperation with The Egyptian Knowledge Bank (EKB). This research received no external funding”.

Data availability Credit Card Fraud Detection Datasets for this research are available in Kaggle repository (<https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud>).

Declarations

Conflict of interest The authors declare no conflict of interest.

Institutional review board statement Not applicable.

Informed consent Not applicable.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

References

1. Nikkel, B. (2020). Fintech forensics: Criminal investigation and digital evidence in financial technologies. *Forensic Science International: Digital Investigation*, 33(1), 1–17.
2. Pachare, S., & Bangal, S. (2023). Cyber security in the fintech industry: Issues, Challenges, and solutions. *Cybersecurity Issues, Challenges, and solutions in the business world* (pp. 1–17). IGI Global.
3. Salisu, S., & Filipov, V. (2023). Blockchain forensics: A modern approach to investigating cybercrime in the age of decentralization. In *Proceedings of the International Conference on Cyber Warfare and Security*, pp. 338–347.
4. Noor, U., Anwar, Z., Amjad, T., & Choo, K. (2019). A machine learning-based fintech cyber threat attribution framework using high-level indicators of compromise. *Future Generation Computer Systems*, 96(3), 227–242.
5. Wongvorachan, T., He, S., & Bulut, O. (2023). A comparison of undersampling, oversampling, and SMOTE methods for dealing with imbalanced classification in educational data mining. *Information*, 14(1), 25–54.
6. Eltayeb, R., Karrar, A., Osman, W., & Mutasim, M. (2023). Handling imbalanced data through resampling: Systematic review. *Indonesian Journal of Electrical Engineering and Informatics*, 11(2), 503–514.
7. Ding, H., Sun, Y., Wang, Z., Huang, N., Shen, Z., & Cui, X. (2023). RGAN-EL: A GAN and ensemble learning-based hybrid approach for imbalanced data classification. *Information Processing & Management*, 60(2), 1–14. 103235
8. De Zarzà, I., De Curtò, J., & Calafate, C. (2023). Optimizing neural networks for imbalanced data. *Electronics*, 12(12), 1–23.
9. Ribeiro, V. (2020). G. Reynoso-Meza (Ed.), Ensemble learning by means of a multi-objective optimization design approach for dealing with imbalanced data sets. *Expert Systems with Applications* 147 1–26 113232.
10. Han, M., Li, A., Gao, Z., Mu, D., & Liu, S. (2023). A survey of multi-class imbalanced data classification methods. *Journal of Intelligent & Fuzzy Systems*, 44(2), 2471–2501.
11. Yang, B., Tian, G., Luttrell, J., Gong, P., & Zhang, C. (2023). A quantum-based oversampling method for classification of highly imbalanced and overlapped data. *Experimental Biology And Medicine*, 248(24), 2500–2513.
12. Hassib, E., El-Desouky, A., Labib, L., El-Kenawy, E., & WOA+BRNN (2020). An imbalanced big data classification framework using Whale optimization and deep neural network. *Soft Computing*, 24(8), 5573–5592.
13. Singh, S., Singh, S., & Kajla, T. (2023). Checking the effectiveness of blockchain application in fraud detection with a systematic literature review approach. *Contemporary Studies of Risks in Emerging Technology*. <https://doi.org/10.1108/978-1-80455-566-820231003>
14. Sarkar, S., Lincy, R., Sasireka, P., & Mittal, S. (2023). Monitor and detect suspicious online transactions. *International Journal of Electronic Security and Digital Forensics*, 15(6), 632–643.
15. Yu, J., Wang, H., Wang, X., Li, Z., Qin, L., Zhang, W., Liao, J., & Zhang, Y. (2023). Group-based fraud detection network on e-commerce platforms. In *Proceedings of the International Conference on Knowledge Discovery and Data Mining*, pp. 5463–5475.
16. Pan, E. (2024). Machine learning in financial transaction fraud detection and prevention. *Transactions on Economics Business and Management Research*, 5(5), 243–249.
17. Rajendran, S., John, A., Suhas, B., & Sahana, B. (2023). Role of ML and DL in detecting fraudulent transactions. *Artificial intelligence for societal issues* (pp. 59–82). Springer International Publishing.

18. Yi, Z., Cao, X., Pu, X., Wu, Y., Chen, Z., Khan, A., Francis, A., & Li, S. (2023). Fraud detection in capital markets: A novel machine learning approach. *Expert Systems With Applications*, 231(1–25), Article 120760.
19. Gao, H., Kou, G., Liang, H., Zhang, H., Chao, X., Li, C. C., & Dong, Y. (2024). Machine learning in business and finance: A literature review and research opportunities. *Financial Innovation*, 10(1), 35–86.
20. Dhone, M., & Nitya, E. (2023). Big data analytics for fraud detection in financial transactions. *Journal of Data Acquisition and Processing*, 38(3), 290–307.
21. Khodabandehlou, S., Golpayegani, A., & FiFrauD (2024). Unsupervised financial fraud detection in dynamic graph streams. *ACM Transactions on Knowledge Discovery from Data*, 18(5), 1–29.
22. Hanae, A., Abdellah, B., Saida, E., & Youssef, G. (2023). End-to-end real-time architecture for fraud detection in online digital transactions. *International Journal of Advanced Computer Science and Applications*, 14(6), 1–19.
23. Dhone, M., & Nitya, E. (2023). Big data analytics for fraud detection in financial transactions. *Journal of Data Acquisition and Processing*, 38(3), 1–15.
24. Chen, C., Lee, C., Huang, S., & Peng, W. (2024). Credit card fraud detection via intelligent sampling and self-supervised learning. *Acm Transactions on Intelligent Systems and Technology*, 15(2), 1–29.
25. Plakandaras, V., Gogas, P., Papadimitriou, T., & Tsamardinos, I. (2022). Credit card fraud detection with automated machine learning systems. *Applied Artificial Intelligence*, 36(1), 1–23. 2086354.
26. Tong, G., & Shen, J. (2023). Financial transaction fraud detector based on imbalance learning and graph neural network. *Applied Soft Computing*, 149(12), 110984.
27. Razaque, A., Frej, M., Bektemyssova, G., Amsaad, F., Almiyani, M., Alotaibi, A., Jhanjhi, N., Amanzholova, S., & Alshammari, M. (2022). Credit card-not-present fraud detection and prevention using big data analytics algorithms. *Applied Sciences*, 13(1), 57.
28. Fakiha, B. (2023). Forensic credit card fraud detection using deep neural network. *Journal of Southwest Jiaotong University*, 58(1), 1–20.
29. Tayebi, M., & Kafhali, E. (2022). Credit card fraud detection based on hyperparameters' optimization using the differential evolution. *International Journal of Information Security and Privacy*, 16(1), 1–21.
30. Tayebi, M., & Kafhali, E. (2021). S. Hyperparameter optimization using genetic algorithms to detect frauds transactions. In *Proceedings of the International Conference on Artificial Intelligence and Computer Vision*, pp. 288–297, Cham: Springer International Publishing.
31. Sikkandar, H., Saroja, S., Suseandhiran, N., & Manikandan, B. (2023). An intelligent approach for anomaly detection in credit card data using bat optimization algorithm. *Inteligencia Artificial*, 26(72), 202–222.
32. Teng, H., Wang, C., Yang, Q., Chen, X., & Li, R. (2024). Leveraging adversarial augmentation on imbalance data for online trading fraud detection. *IEEE Transactions on Computational Social Systems*, 11(2), 1602–1614.
33. El Hlouli, F., Riffi, J., Mahraz, M., Yahyaouy, A., El Fazazy, K., & Tairi, H. (2024). Credit card fraud detection: Addressing imbalanced datasets with a Multi-phase approach. *SN Computer Science*, 5(1), 173.
34. Liu, B., Chen, X., & Yu, K. (2023). Online transaction fraud detection system based on machine learning. *Journal of Physics: Conference Series*, 2023(1):1–7, IOP Publishing.
35. Innan, N., Khan, M., & Bennai, M. (2024). Financial fraud detection: A comparative study of quantum machine learning models. *International Journal of Quantum Information*, 22(02), 1–30. 2350044
36. Vanini, P., Rossi, S., Zvizdic, E., & Domenig, T. (2023). Online payment fraud: From anomaly detection to risk management. *Financial Innovation*, 9(1), 1–17. 66.
37. Seera, M., Lim, C., Kumar, A., Dhamotharan, L., & Tan, K. (2024). An intelligent payment card fraud detection system. *Annals of Operations Research*, 334(1), 445–467.
38. Gazi, M. (2023). Exploring machine learning techniques for fraud detection in financial transactions. *Chinese Journal of Geotechnical Engineering*, 45(10), 25–32.
39. Belyakov, S., Karpov, S., & Zalilov, E. (2020). Detecting Fraudulent Transactions Using a Machine Learning Algorithm. In *Proceedings of the Third International Conference for Futuristic Trends in Network and Communication Technologies*, Russia, October 14–16, pp. 46–55, Springer Singapore.

40. Wang, H., Zheng, J., Carvajal-Roca, I., Chen, L., & Bai, M. (2023). Financial Fraud Detection Based on Deep Learning: Towards Large-Scale Pre-training Transformer Models. In *Proceedings of the China Conference on Knowledge Graph and Semantic Computing*, pp. 163–177, Singapore: Springer Nature Singapore.
41. Al-Sayyed, R., Alhenawi, E., Alazzam, H., Wrikat, A., & Suleiman, D. (2024). Mobile money fraud detection using data analysis and visualization techniques. *Multimedia Tools and Applications*, 83(6), 17093–17108.
42. Thimonier, H., Popineau, F., Rimmel, A., Doan, B., & Daniel, F. (2023). Comparative evaluation of anomaly detection methods for fraud detection in online credit card Payments, arxiv Preprint arxiv:2312.13896.
43. Cherif, A., Badhib, A., Ammar, H., Alshehri, S., Kalkatawi, M., & Imine, A. (2023). Credit card fraud detection in the era of disruptive technologies: A systematic review. *Journal of King Saud University-Computer and Information Sciences*, 35(1), 145–174.
44. Podile, V., Dhoke, S., Guha, S., Ahmed, F., Hariitha, T., Abhinav, V., & Anirudh, M. (2024). Leveraging Distribution Systems for Advanced Fraud Detection and Prevention in Finance. In Book, *Meta Heuristic Algorithms for Advanced Distributed Systems*, pp. 211–228.
45. Motie, S., & Raaheemi, B. (2023). Financial fraud detection using graph neural networks: A systematic review. *Expert Systems with Applications*, 240(4), 122156.
46. Hu, J., Hu, R., Wang, Z., Li, D., Wu, J., Ren, L., Zang, Y., Huang, Z., & Wang, M. (2023). Collaborative Fraud Detection: How Collaboration Impacts Fraud Detection. In *Proceedings of the 31st ACM International Conference on Multimedia*, pp. 8891–8899.
47. Tao, Z., & Chao, J. F. (2023). Fraud and Anomaly Detection Techniques: A Literature Review. In *Proceedings of the 4th International Conference on Computer Science and Management Technology*, pp. 634–641.
48. Viswanatha, V., Ramachandra, A., Deeksha, V., & Ranjitha, R. (2023). Online fraud detection using machine learning approach. *International Journal of Engineering and Management Research*, 13(4), 45–57.
49. Shahana, T., Lavanya, V., & Bhat, A. (2023). State of the art in financial statement fraud detection: A systematic review. *Technological Forecasting And Social Change*, 192(1), 1–24.
50. Aftab, A., Shahzad, I., Anwar, M., Sajid, A., & Anwar, N. (2023). Fraud detection of credit cards using supervised machine learning. *Pakistan Journal of Emerging Science and Technologies*, 4(3), 1–4.
51. Hanae, A., Youssef, G., & Saida, E. (2023). Analysis of Banking Fraud Detection Methods through Machine Learning Strategies in the Era of Digital Transactions. In *Proceedings of the IEEE Congress on Information Science and Technology*, pp. 105–110.
52. Braytee, A., Hussain, F. K., Anaissi, A., & Kennedy, P. J. ABC-sampling for balancing imbalanced datasets based on artificial bee colony algorithm. In 2015 IEEE 14th international conference on machine learning and applications (ICMLA) 2015 Dec 9 (pp. 594–599). IEEE.
53. Ajilisa, O. A., Jagathyraj, V. P., & Sabu, M. K. (2021). A Novel Clustering Based Undersampling Algorithm for Imbalanced Data Sets Using Artificial Bee Colony Algorithm. In *Innovations in Bio-Inspired Computing and Applications: Proceedings of the 11th International Conference on Innovations in Bio-Inspired Computing and Applications (IBICA 2020) held during December 16–18, 2020* 11 (pp. 32–42). Springer International Publishing.
54. Dastidar, K. G., Caelen, O., & Granitzer, M. (2024). Machine learning methods for credit card fraud detection: A survey. *Ieee Access : Practical Innovations, Open Solutions*. Oct 28.
55. Kholidy, H. A. (2021). Detecting impersonation attacks in cloud computing environments using a centric user profiling approach. *Future Generation Computer Systems*, 117, 299–320.
56. Sreekala, K., Sridivya, R., Rao, N. K., Mandal, R. K., Moses, G. J., & Lakshmanarao, A. A hybrid K-means and ML Classification Approach for Credit Card Fraud Detection. In 2024 3rd International Conference for Innovation in Technology (INOCON) 2024 Mar 1 (pp. 1–5). IEEE.
57. Jebaraj, N. S., Shekhawat, J., & Gupta, R. An Overview of Clustering Algorithms for Credit Card Fraud Detection. In 2024 International Conference on Optimization Computing and Wireless Communication (ICOCWC) 2024 Jan 29 (pp. 1–6). IEEE.
58. Khan, S., & Palaniswamy, B. (2024). Enhancing Credit Card Fraud Detection: A Comparative Analysis of Artificial Neural Networks, k-NN, and Naive Bayes Classifiers. In *IEEE Students Conference on Engineering and Systems (SCES) 2024 Jun 21* (pp. 1–6). IEEE.
59. Shrivastava, V., & Chaturvedi, A. K. (2024). A review on intrusion detection system for distributed network based on machine learning. *Journal of Integrated Science and Technology*, 12(2), 739.

60. Hijazi, A., Alhafez, N., & Al-khayat, I. (2024). An adaptive distributed intrusion detection system in local network: Hybrid classification methods. *Journal of Intelligent Systems & Internet of Things*, 12(1), 129–140.
61. Khraisat, A., Alazab, A., Singh, S., Jan, T., Jr., & Gomez, A. (2024). Survey on federated learning for intrusion detection system: Concept, architectures, aggregation strategies, challenges, and future directions. *Acm Computing Surveys*, 57(1), 1–38.
62. Ahmadi, S. (2024). Network intrusion detection in cloud environments: A comparative analysis of approaches. *International Journal of Advanced Computer Science and Applications*, 15(3), 1–8.
63. Diana, L., Dini, P., & Paolini, D. (2025). Overview on intrusion detection systems for computers networking security. *Computers*, 14(3), 1–44.
64. Darwish, S. M. (2020). An intelligent credit card fraud detection approach based on semantic fusion of two classifiers. *Soft Computing*, 24(2), 1243–1253.

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Authors and Affiliations

Saad M. Darwish¹  · Amr Ibrahim Salama² · Adel A. Elzoghbi³ · Noha A. El-Shoafy⁴

✉ Saad M. Darwish
saad.darwish@alexu.edu.eg

¹ Department of Information Technology, Institute of Graduate Studies and Research, Alexandria University, 163 Horreya Avenue, El Shatby, P.O. Box 832, Alexandria 21526, Egypt

² CEO, Sentinel Shield Solutions- FZCO, Office 1701 Prime Tower, Business Bay, Dubai 25314, UAE

³ College of Computers and Data Science, Alexandria University, Postal code 21568, Alexandria, Egypt

⁴ Computer Science Department, Faculty of Computers and Artificial Intelligence, Matrouh University, Matrouh 51511, Egypt